

Java Full Stack Development

1. Testing





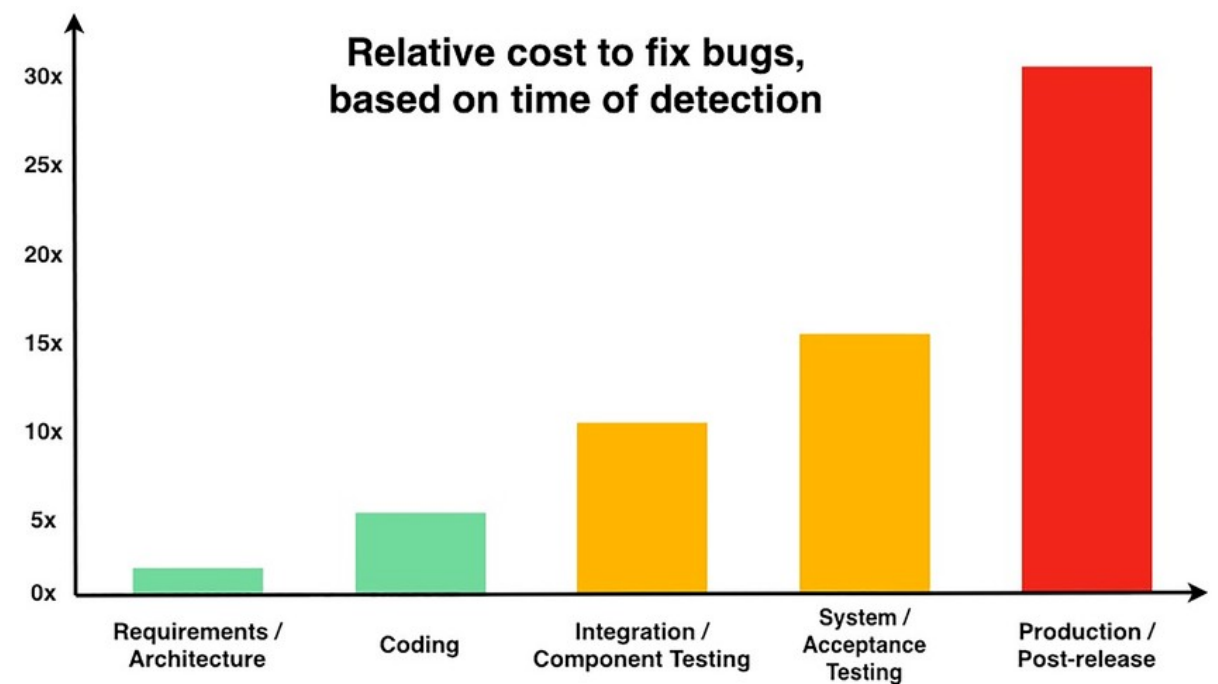
Module Topics

- The Role of Testing
- JUnit 5 Fundamentals
- Mocking with Mockito
- Spring Boot Integration Testing

The Role of Testing

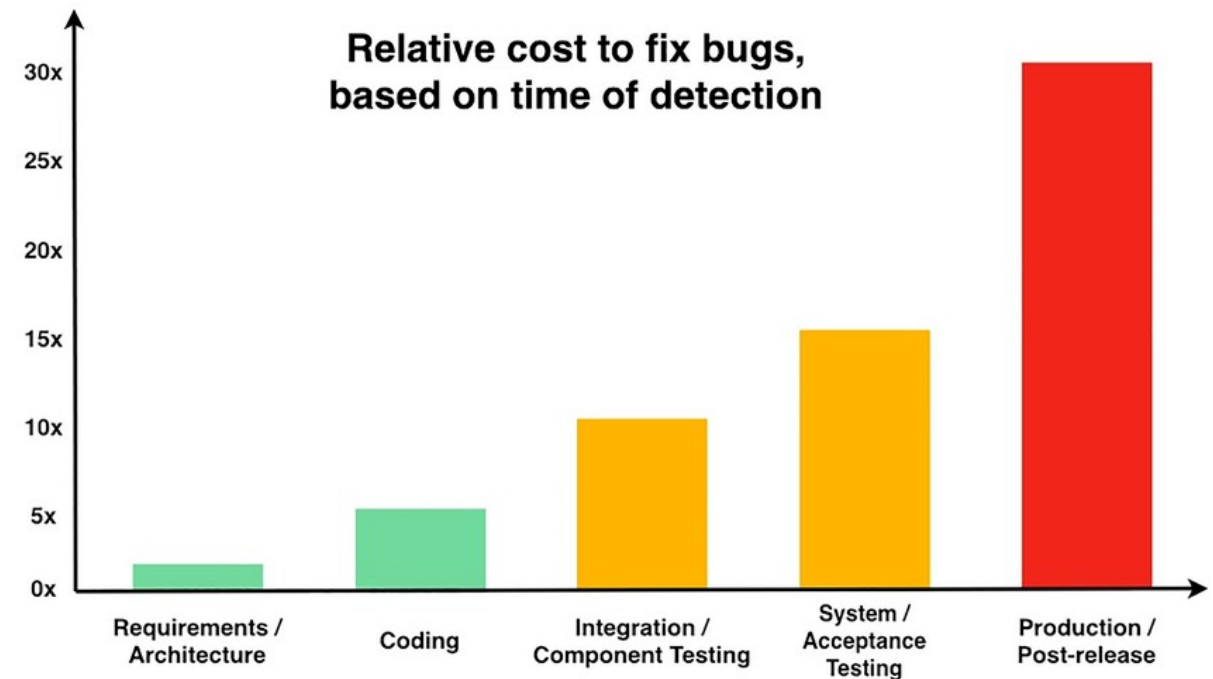
The Economics of Bugs

- Bugs get more expensive over time
 - Bug found at design time
 - Minutes to fix, conversation to update
 - Bug found in development
 - Hours to fix, code change in progress
 - Bug found in QA
 - Days to fix, regression cycle, retest
 - Bug found in production
 - Weeks to fix, customer impact, possible regulatory disclosure



The Economics of Bugs

- Tests shift bug discovery left
 - Toward development, where bugs are cheap
 - Remediation costs increase exponentially the further in the development cycle bugs are discovered
 - Cost estimates are supported by decades of research, starting in the 1980s
 - Bugs have opportunity costs
 - Time spent debugging production issues is time not spent building new features
 - Over a fiscal year, this is a measurable productivity drag, even before you count direct costs.
 - The cost of writing a test is amortized over every future regression it prevents
 - A well-written test costs once and pays back every time someone changes nearby code.

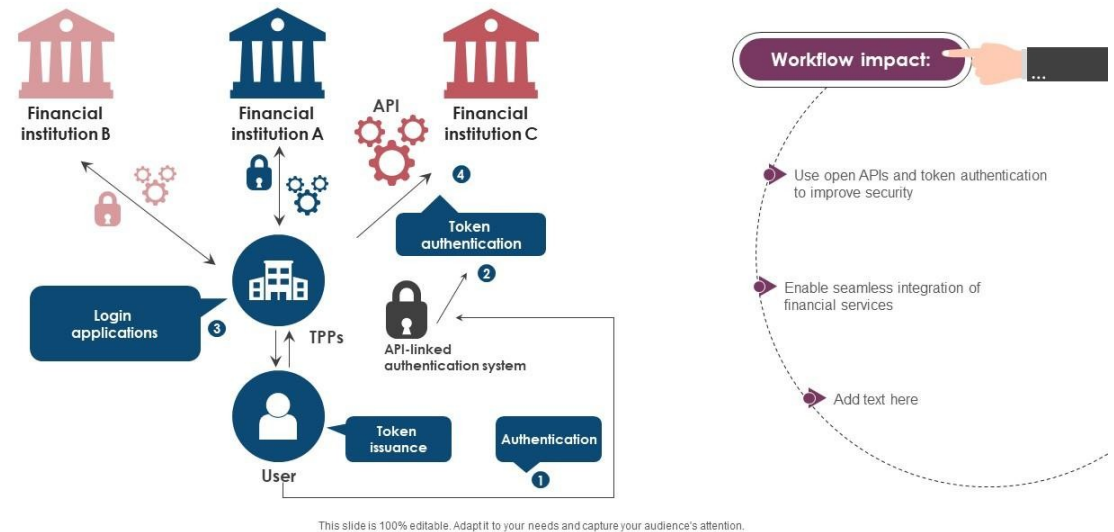


Testing in a Regulated Industry

- Regulatory testing requirements
 - Audit trails
 - Regulators want evidence that changes were tested before deploy
 - Change management
 - Documented test results are required for QA gate release approval
 - Due diligence
 - "We tested this thoroughly" is a defensible position
 - "We tested it the way the developer felt was sufficient" is not
 - Operational risk frameworks
 - Inadequate testing is itself a recordable risk

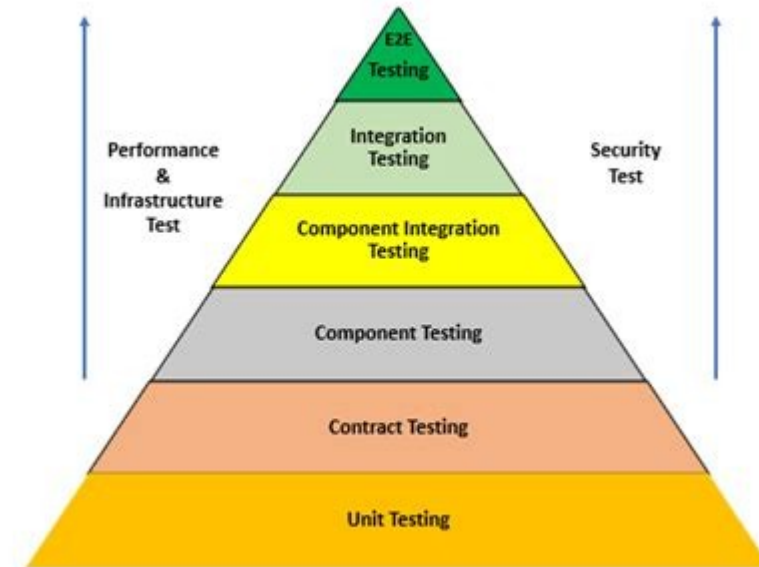
Open API banking regulatory framework

The purpose of this slide is to highlight the strategic adoption of open APIs in banking and financial services industry. It includes key insights such as using open APIs and token authentication to improve security, etc.



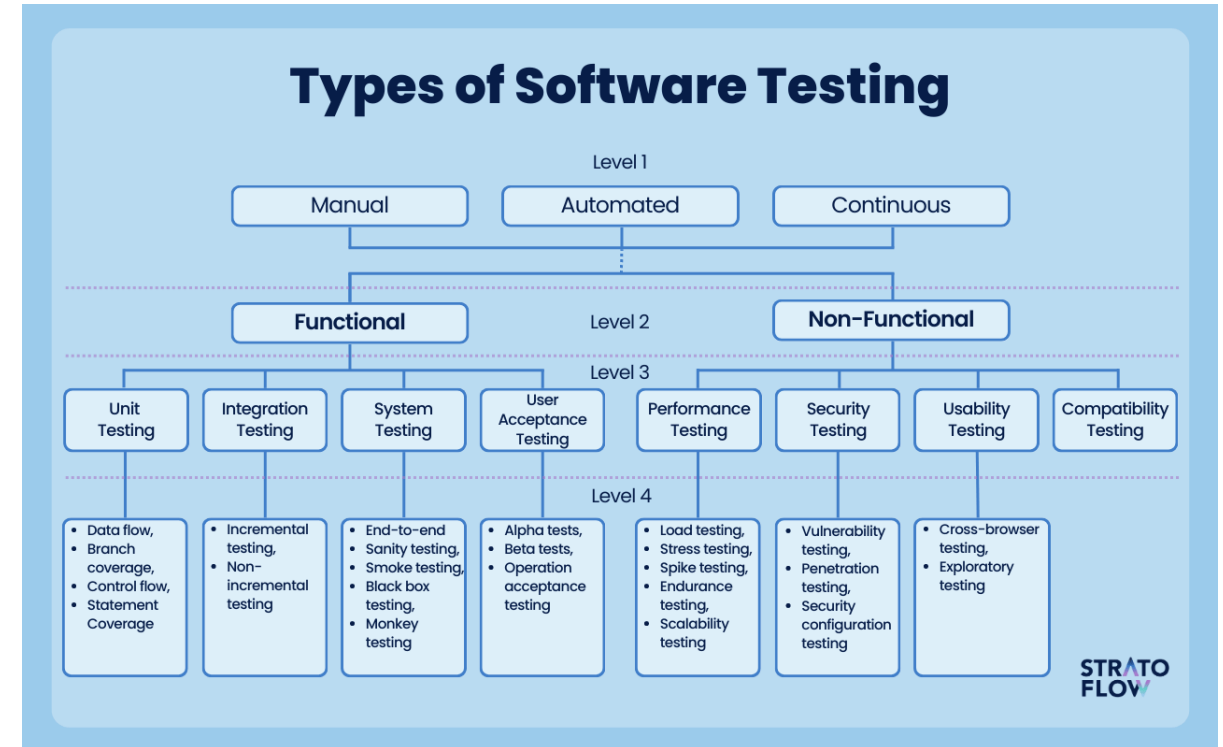
Testing Pyramid

- Most common levels of testing
 - Unit tests
 - Many, fast, cheap; one class or function in isolation
 - Integration tests
 - Fewer, slower, more expensive; multiple components together
 - End-to-end tests
 - Few, slow, fragile
 - Because we test the whole system through the UI
 - The higher we are on the pyramid
 - The more complex and expensive the testing
 - The more intricate the test scaffolding
 - The more we rely on lower levels being done right



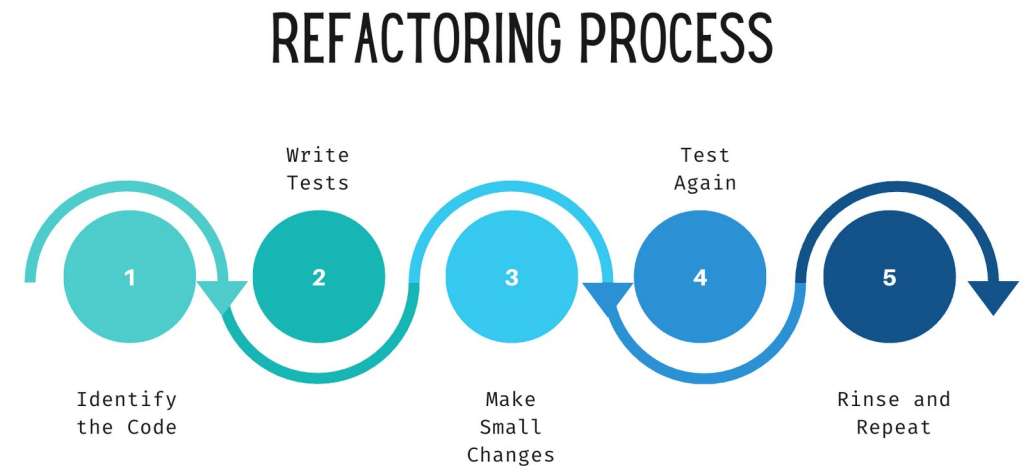
Testing Limits

- What testing does not do
 - Tests do not prove correctness
 - They show the test cases passed
 - Badly designed test cases give false assurances that there are no bugs in the system
 - Tests do not replace code review
 - A human still has to think about design and security
 - Tests do not replace security review
 - Functional correctness and security are different concerns
 - Tests do not catch bugs you did not anticipate
 - You cannot test for the unknown unknowns
 - High coverage with weak assertions is worse than honest lower coverage



Refactoring

- Tests as scaffolding for refactoring
 - Tests let you change code with confidence
 - Without tests, every change is a gamble
 - Did I break something?
 - With tests, refactoring is safe
 - The tests catch you when you slip
 - This is often the biggest day-to-day value of a test suite



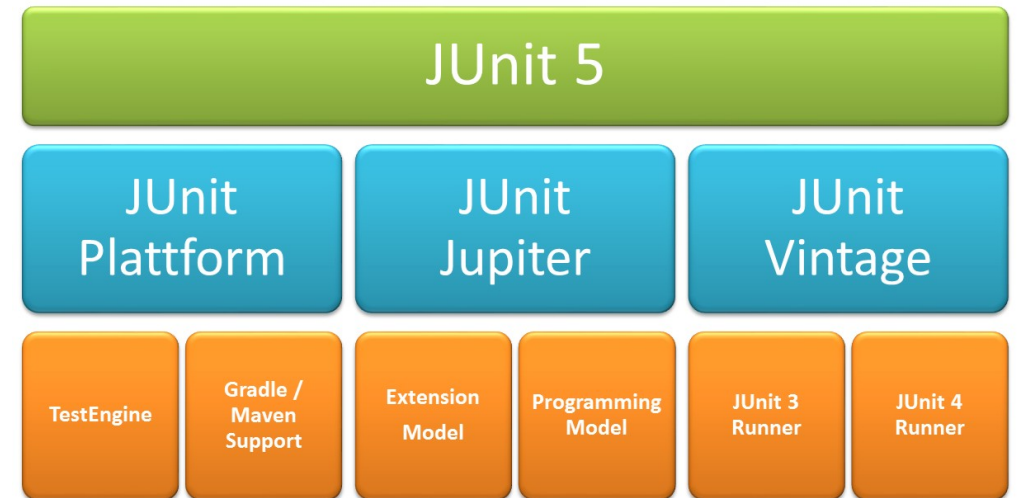
Refactoring

- Tests for refactoring
 - Need to be automated
 - Otherwise development is impeded
 - Too much development time is spent on writing and running tests
 - Tests ensure functional correctness
 - While code structure is optimized
 - Ensures that technical debt is paid off immediately
 - Reduces downstream maintenance load



Tooling

- Comparison to C#
 - JUnit 5 maps to NUnit / xUnit / MSTest
 - The test framework
 - Mockito maps to Moq
 - The mocking library
 - AssertJ maps to FluentAssertions
 - Spring Boot Test maps to WebApplicationFactory / TestServer
 - Integration test infrastructure
 - The concepts are congruent
 - Only the syntax differs



Inherited Code

- Much of the legacy code you'll test was written without tests
 - Banks have decades of code written before automated testing was standard
 - Adding tests to legacy code is harder than writing tests with the code
 - The strategy is to add tests when you change code, not in a heroic top-down sweep
 - Imperfect coverage that grows is better than perfect coverage that doesn't exist

```
public abstract class AbstractCollection implements Collection {  
    public void addAll(AbstractCollection c) {  
        if (c instanceof Set) {  
            Set s = (Set)c;  
            for (int i=0; i < s.size(); i++) {  
                if (!contains(s.elementAt(i))) {  
                    add(s.elementAt(i));  
                }  
            }  
        }  
        else if (c instanceof List) {  
            List l = (List)c;  
            for (int i=0; i < l.size(); i++) {  
                if (!contains(l.get(i))) {  
                    add(l.get(i));  
                }  
            }  
        }  
        else if (c instanceof Map) {  
            Map m = (Map)c;  
            for (int i=0; i<m.size(); i++)  
                add(m.keys[i], m.values[i]);  
        }  
    }  
}
```

The diagram highlights several code quality issues in the provided Java code snippet:

- Duplicated Code:** Indicated by a blue oval around the `if (c instanceof Set)` block and a purple oval around the `if (c instanceof List)` block, with arrows pointing to the label.
- Alternative Classes with Different Interfaces:** Indicated by a green oval around the `if (!contains(s.elementAt(i)))` and `if (!contains(l.get(i)))` conditions, with an arrow pointing to the label.
- Switch Statement:** Indicated by a red oval around the `else if (c instanceof Map)` block, with an arrow pointing to the label.
- Inappropriate Intimacy:** Indicated by a red oval around the `add(m.keys[i], m.values[i]);` line, with an arrow pointing to the label.
- Long Method:** Indicated by a black oval around the entire `addAll` method, with an arrow pointing to the label.

Inherited Code

- When to write a test for legacy code
 - When you fix a bug
 - Write a test that would have caught it
 - Coverage goes up by one test, and you have a regression test for the bug
 - When you add a feature
 - Write tests for the new code
 - The new code is testable because you wrote it that way
 - The old code remains as it was.

```
public abstract class AbstractCollection implements Collection {  
    public void addAll(AbstractCollection c) {  
        if (c instanceof Set) {  
            Set s = (Set)c;  
            for (int i=0; i < s.size(); i++) {  
                if (!contains(s.getElementAt(i))) {  
                    add(s.getElementAt(i));  
                }  
            }  
        }  
        else if (c instanceof List) {  
            List l = (List)c;  
            for (int i=0; i < l.size(); i++) {  
                if (!contains(l.get(i))) {  
                    add(l.get(i));  
                }  
            }  
        }  
        else if (c instanceof Map) {  
            Map m = (Map)c;  
            for (int i=0; i<m.size(); i++)  
                add(m.keys[i], m.values[i]);  
        }  
    }  
}
```

The diagram highlights several code quality issues in the `addAll` method:

- Duplicated Code:** Points to the `if (!contains(s.getElementAt(i)))` and `add(s.getElementAt(i));` lines in the Set branch.
- Duplicated Code:** Points to the `if (!contains(l.get(i)))` and `add(l.get(i));` lines in the List branch.
- Alternative Classes with Different Interfaces:** Points to the `if (!contains(l.get(i)))` line in the List branch.
- Switch Statement:** Points to the `if (c instanceof Set)` and `else if (c instanceof List)` and `else if (c instanceof Map)` branches.
- Inappropriate Intimacy:** Points to the `m.keys[i]` and `m.values[i]` access in the Map branch.
- Long Method:** Points to the entire `addAll` method.

Inherited Code

- When you refactor a hard-to-test class
 - Take small steps
 - Extract a method
 - Give it dependencies it can be supplied with
 - Write a test for the extracted piece
 - The class as a whole is still hard to test
 - One piece of it now is not.

```
public abstract class AbstractCollection implements Collection {  
    public void addAll(AbstractCollection c) {  
        if (c instanceof Set) {  
            Set s = (Set)c;  
            for (int i=0; i < s.size(); i++) {  
                if (!contains(s.elementAt(i))) {  
                    add(s.elementAt(i));  
                }  
            }  
        } else if (c instanceof List) {  
            List l = (List)c;  
            for (int i=0; i < l.size(); i++) {  
                if (!contains(l.get(i))) {  
                    add(l.get(i));  
                }  
            }  
        } else if (c instanceof Map) {  
            Map m = (Map)c;  
            for (int i=0; i<m.size(); i++)  
                add(m.keys[i], m.values[i]);  
        }  
    }  
}
```

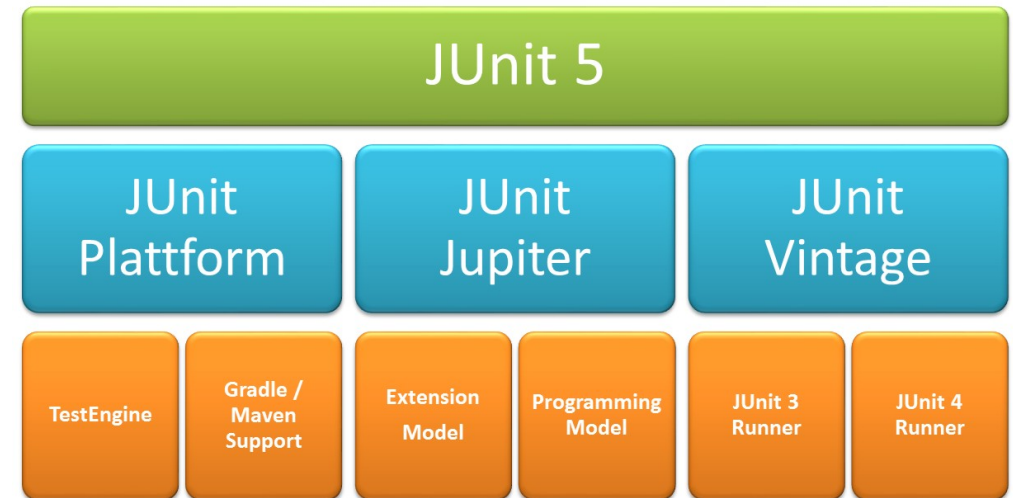
The diagram shows a Java code snippet for an abstract class `AbstractCollection` implementing the `Collection` interface. The `addAll` method is annotated with several callouts:

- Duplicated Code**: Points to the `if (c instanceof Set)` block and the `if (!contains(s.elementAt(i)))` condition.
- Duplicated Code**: Points to the `if (!contains(l.get(i)))` condition in the `List` block.
- Alternative Classes with Different Interfaces**: Points to the `if (!contains(l.get(i)))` condition in the `List` block.
- Switch Statement**: Points to the `if (c instanceof Set)` block.
- Inappropriate Intimacy**: Points to the `m.keys[i]` and `m.values[i]` access in the `Map` block.
- Long Method**: Points to the entire `addAll` method.

JUnit 5 Fundamentals

JUnit 5 and JUnit 4

- What's different from JUnit 4
 - Modular
 - Separate engine, API, parameter resolution
 - Java 8 baseline
 - Lambdas in assertions, streams in test data
 - Native parameterized tests
 - No separate runner needed
 - Better extension model
 - `@ExtendWith` for things like Mockito and Spring
 - Default visibility
 - Tests can be package-private (no need for public)



Test Method

- Elements of a test method
 - `@Test` marks the method as a test
 - Method body follows AAA
 - Arrange
 - Set up the inputs and any dependencies the unit needs.
 - Act
 - Call the one method or operation under test
 - If you find yourself doing two things in this section, you're testing two things
 - Then you should probably split into two tests.
 - Assert
 - Verify the outcome.
 - One assertion per concept

```
@Test
void transferReducesSourceAccountBalance() {
    // Arrange
    Account from = new Account("ACC-001", new BigDecimal("100.00"));
    Account to = new Account("ACC-002", new BigDecimal("0.00"));

    // Act
    transferService.transfer(from, to, new BigDecimal("30.00"));

    // Assert
    assertThat(from.getBalance()).isEqualTo(new BigDecimal("70.00"));
}
```

Test Method

- Good test names
 - Describe behavior, not methods
 - Bad: TestTransfer1, testTransferMethod, testTransfer_2
 - Better: testTransferUpdatesBalances
 - Best:
transferReducesSourceAccountBalance or
transferRejectsAmountGreaterThanBalance
 - The test name should tell you what's being verified, without reading the body
 - When a test fails
 - The name appears in the report
 - A good name increases readability

```
@Test
void transferReducesSourceAccountBalance() {
    // Arrange
    Account from = new Account("ACC-001", new BigDecimal("100.00"));
    Account to = new Account("ACC-002", new BigDecimal("0.00"));

    // Act
    transferService.transfer(from, to, new BigDecimal("30.00"));

    // Assert
    assertThat(from.getBalance()).isEqualTo(new BigDecimal("70.00"));
}
```

Lifecycle Annotations

- Setup and teardown hooks
 - `@BeforeAll` and `@AfterAll` must be static
 - `BeforeAll` executed once before tests
 - Does general setup that applies to all tests
 - `AfterAll` executed once at the end of testing
 - Usually tears down what `BeforeAll` sets up
 - Most setup belongs in `@BeforeEach`, not `@BeforeAll`
 - `BeforeEach` runs before each test
 - Sets up the test baseline
 - For example, setting bank account balance to a specific value
 - `AfterEach` runs after each test
 - Undoes any side effects of running the test

```
class TransferServiceTest {  
  
    @BeforeEach  
    void setUp() {  
        // Runs before EVERY test method  
        transferService = new TransferService(...);  
    }  
  
    @AfterEach  
    void tearDown() {  
        // Runs after EVERY test method  
    }  
  
    @BeforeAll  
    static void initializeSuite() {  
        // Runs ONCE before all tests in this class  
    }  
  
    @AfterAll  
    static void cleanupSuite() {  
        // Runs ONCE after all tests in this class  
    }  
}
```

Assertions

- Common patterns
 - `assertThrows`
 - Tests that calling some code raises a specific exception
 - If the code does not throw, the test fails
 - If the code throws a different exception type, the test fails
 - If the code throws the expected exception, `assertThrows` returns it
 - `assertAll`
 - Runs multiple assertions and reports all failures at once
 - Without `assertAll`, the first failed assertion stops the test, and you don't see whether subsequent assertions would also have failed
 - With `assertAll`, you see all failures together
 - Use it for related assertions where you want a complete picture.

```
// Basic
assertEquals(expected, actual);
assertNotNull(value);
assertTrue(condition);

// Exception-throwing code
assertThrows(InsufficientFundsException.class, () -> transfer.execute());

// Multiple related assertions, all reported even if one fails
assertAll(
    () -> assertEquals(70, from.getBalance()),
    () -> assertEquals(30, to.getBalance()));

// Time-bounded operations
assertTimeout(Duration.ofMillis(500), () -> service.expensiveOperation())

// Confirmation that nothing was thrown
assertDoesNotThrow(() -> validator.validate(input));
```

Assertions

- Common patterns

- `assertTimeout`

- For testing operations that should complete within a bounded time
 - It does not abort the operation
 - If it takes too long; it just fails the test
 - Useful for catching performance regressions in critical paths

- `assertDoesNotThrow`

- For the rare case where the test's purpose is "confirming no exception is raised"
 - Most of the time, the absence of an exception is implicit
 - But sometimes you want to be explicit, especially when refactoring code that previously threw

```
// Basic
assertEquals(expected, actual);
assertNotNull(value);
assertTrue(condition);

// Exception-throwing code
assertThrows(InsufficientFundsException.class, () -> transfer.execute());

// Multiple related assertions, all reported even if one fails
assertAll(
    () -> assertEquals(70, from.getBalance()),
    () -> assertEquals(30, to.getBalance()));

// Time-bounded operations
assertTimeout(Duration.ofMillis(500), () -> service.expensiveOperation())

// Confirmation that nothing was thrown
assertDoesNotThrow(() -> validator.validate(input));
```

AssertJ

- Third-party assertion library
 - Recommend using over JUnit's built-in assertions
 - Spring Boot projects already include AssertJ on the test classpath
 - You do not need to add a dependency.
 - The main argument for AssertJ is readability
 - `assertThat(x).isEqualTo(y)`
 - Reads left-to-right naturally
 - `assertEquals(y, x)` has the expected value coming first, opposite to how you'd write it in natural language

```
// JUnit assertions
assertEquals("ACC-001", account.getAccountNumber());
assertTrue(account.getBalance().compareTo(BigDecimal.ZERO) > 0);

// AssertJ assertions
assertThat(account.getAccountNumber()).isEqualTo("ACC-001");
assertThat(account.getBalance()).isPositive();

// Chained assertions for richer checks
assertThat(account)
    .isNotNull()
    .extracting(Account::getStatus)
    .isEqualTo(AccountStatus.ACTIVE);

// Exception assertions read as English
assertThatThrownBy(() -> service.transfer(from, to, tooMuch))
    .assertInstanceOf(InsufficientFundsException.class)
    .hasMessageContaining("ACC-001");
```


AssertJ

- Chained-assertion style on the slide
 - Productivity boost for tests that verify object state
 - Instead of writing five separate assertions, you chain them into one expression that reads like a description of the object
 - The exception assertion in the last example is genuinely better than JUnit's `assertThrows`
 - It assertively checks the type AND any properties of the exception in one expression

```
// JUnit: works but verbose
InsufficientFundsException ex = assertThrows(
    InsufficientFundsException.class,
    () -> service.transfer(from, to, tooMuch));
assertTrue(ex.getMessage().contains("ACC-001"));

// AssertJ: same intent, half the code
assertThatThrownBy(() -> service.transfer(from, to, tooMuch))
    .assertInstanceOf(InsufficientFundsException.class)
    .hasMessageContaining("ACC-001");
```

Display Names

- Makes tests readable
 - `@DisplayName` provides human-readable labels in test reports
 - `@Nested` groups related tests, including their setup
 - Lets you create inner classes that group related tests.
 - Each inner class can have its own `@BeforeEach` for setup that's only relevant to that group
 - The result is a test class that's structured the way you'd structure a documentation page
 - Outer class names the unit under test
 - inner classes name the conditions or scenarios
 - Methods name the specific behaviors

```
@DisplayName("Transfer service")
class TransferServiceTest {

    @Nested
    @DisplayName("when source account has insufficient funds")
    class WhenInsufficientFunds {

        @Test
        @DisplayName("throws InsufficientFundsException")
        void throwsInsufficientFundsException() { ... }

        @Test
        @DisplayName("does not modify either account")
        void doesNotModifyEitherAccount() { ... }
    }

    @Nested
    @DisplayName("when transfer is valid")
    class WhenTransferIsValid {

        @Test
        @DisplayName("reduces source balance")
        void reducesSourceBalance() { ... }
    }
}
```

Parameterized Tests

- Parameterized tests
 - Run the same test method multiple times with different inputs
 - The example with blank account numbers is the canonical use case
 - The validator should reject blank input regardless of what kind of whitespace is in the string
 - Without parameterization you'd write five near-identical tests, one per whitespace variant
 - With `@ValueSource`, you write one test method and JUnit runs it five times, once per input

```
@ParameterizedTest
@ValueSource(strings = { "", " ", "   ", "\t", "\n" })
void rejectsBlankAccountNumber(String input) {
    assertThatThrownBy(() -> validator.validate(input))
        .isInstanceOf(ValidationException.class);
}

@ParameterizedTest
@CsvSource({
    "100.00, 30.00, 70.00",
    "100.00, 0.01, 99.99",
    "100.00, 100.00, 0.00"
})
void transferReducesSourceCorrectly(BigDecimal start, BigDecimal amount, BigDecimal expected) {
    Account from = new Account("ACC-001", start);
    transferService.transfer(from, accountB, amount);
    assertThat(from.getBalance()).isEqualTo(expected);
}
```

Parameterized Tests

- Parameterized tests
 - `@ValueSource`
 - Works for one parameter only
 - A list of strings, ints, longs, etc.
 - For multi-parameter tests, use `@CsvSource`
 - Each row of the CSV becomes one test invocation; the columns map to method parameters in order
 - `@MethodSource`
 - For cases where your test data needs to be constructed dynamically
 - Complex objects, calculated values, data loaded from a file
 - The annotation references a method that returns a Stream of arguments

```
@ParameterizedTest
@ValueSource(strings = { "", " ", " ", "\t", "\n" })
void rejectsBlankAccountNumber(String input) {
    assertThatThrownBy(() -> validator.validate(input))
        .isInstanceOf(ValidationException.class);
}

@ParameterizedTest
@CsvSource({
    "100.00, 30.00, 70.00",
    "100.00, 0.01, 99.99",
    "100.00, 100.00, 0.00"
})
void transferReducesSourceCorrectly(BigDecimal start, BigDecimal amount, BigDecimal expected) {
    Account from = new Account("ACC-001", start);
    transferService.transfer(from, accountB, amount);
    assertThat(from.getBalance()).isEqualTo(expected);
}
```

Example

- Shows
 - `@DisplayName`: human-readable class label
 - `@BeforeEach` for setup
 - `@Test` to mark the test method
 - `@DisplayName` on the method for a clear failure label
 - AAA structure with the body broken into Arrange / Act+Assert
 - Combined here, since `assertThatThrownBy` exercises both
 - AssertJ for the fluent assertion
 - Behavior-focused method name
 - `transferRejectsAmountGreaterThanBalance`

```
@DisplayName("Transfer service")
class TransferServiceTest {

    private TransferService transferService;

    @BeforeEach
    void setUp() {
        transferService = new TransferService();
    }

    @Test
    @DisplayName("transfer rejects amounts greater than account balance")
    void transferRejectsAmountGreaterThanBalance() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));

        assertThatThrownBy(
            () -> transferService.transfer(from, to, new BigDecimal("150.00")))
            .assertInstanceOf(InsufficientFundsException.class)
            .hasMessageContaining("ACC-001");
    }
}
```

F.I.R.S.T.

- Properties of good tests
 - Fast
 - Thousands of tests should run in under a minute
 - Independent
 - Tests do not depend on order, on each other, or on shared mutable state
 - Repeatable
 - Same input, same result, every time, in any environment
 - Self-validating
 - Tests pass or fail; no human inspection required
 - Timely
 - Written close in time to the production code, not months later

```
@DisplayName("Transfer service")
class TransferServiceTest {

    private TransferService transferService;

    @BeforeEach
    void setUp() {
        transferService = new TransferService();
    }

    @Test
    @DisplayName("transfer rejects amounts greater than account balance")
    void transferRejectsAmountGreaterThanBalance() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));

        assertThatThrownBy(
            () -> transferService.transfer(from, to, new BigDecimal("150.00")))
            .isInstanceOf(InsufficientFundsException.class)
            .hasMessageContaining("ACC-001");
    }
}
```


Style Guide

- Best practices
 - Behavior-focused names
 - AAA structure
 - AssertJ assertions
 - `@Nested` for organization
 - One logical assertion per test
 - Multiple physical assertions verifying the same outcome are fine
 - “Pass” condition may have multiple sub-conditions

```
@DisplayName("Transfer service")
class TransferServiceTest {

    private TransferService transferService;

    @BeforeEach
    void setUp() {
        transferService = new TransferService();
    }

    @Test
    @DisplayName("transfer rejects amounts greater than account balance")
    void transferRejectsAmountGreaterThanBalance() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));

        assertThatThrownBy(
            () -> transferService.transfer(from, to, new BigDecimal("150.00")))
            .assertInstanceOf(InsufficientFundsException.class)
            .hasMessageContaining("ACC-001");
    }
}
```

Style Guide

- Problematic
 - Test names that mirror method names
 - Testing implementation details
 - Tests with multiple Act steps
 - Should be split into multiple tests
 - Comments explaining what the test does
 - The test name should be self-explanatory
 - Thread.sleep(1000) to "wait for things"
 - Use proper synchronization or async testing instead

```
@DisplayName("Transfer service")
class TransferServiceTest {

    private TransferService transferService;

    @BeforeEach
    void setUp() {
        transferService = new TransferService();
    }

    @Test
    @DisplayName("transfer rejects amounts greater than account balance")
    void transferRejectsAmountGreaterThanBalance() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));

        assertThatThrownBy(
            () -> transferService.transfer(from, to, new BigDecimal("150.00")))
            .isInstanceOf(InsufficientFundsException.class)
            .hasMessageContaining("ACC-001");
    }
}
```

Mocking with Mockito

Mockito

- Problems with real dependencies
 - Slow
 - A database call adds milliseconds
 - Multiplied across hundreds of tests results in significant slowdowns
 - Non-deterministic
 - Network calls fail intermittently, time-dependent code returns different values
 - Expensive to set up
 - Spinning up a database or external service for every test is overkill for unit-level checks



Mockito

- Problems with real dependencies
 - Out of your control
 - Third-party APIs may have rate limits, may be down, may charge per call
 - Hard to drive into edge cases
 - Making a real database simulate "connection dropped mid-transaction" is much harder than telling a mock to throw
 - Confounding errors
 - Problems in dependencies can give false positives for tests



Mockito

- Terminology

- Dummy
 - Placeholder object, never actually used; just satisfies a parameter
- Stub
 - Returns canned answers to calls but has no logic, no verification
- Fake
 - Has working implementation, but unsuitable for production like an in-memory DB



Mockito

- Terminology

- Mock

- Pre-programmed with expectations
 - Verifies that calls happened correctly

- Spy

- Wraps a real object
 - You can override some methods, real ones still work



Mockito Basics

- Three basic operations
 - `mock(T.class)`
 - Creates an instance with all methods returning defaults
 - `when(...).thenReturn(...)`
 - Programs return values for specific calls
 - `verify(...)`
 - Asserts that specific calls did or did not happen

```
import static org.mockito.Mockito.*;

// 1. Create a mock
AccountRepository repo = mock(AccountRepository.class);

// 2. Stub its behavior
when(repo.findById("ACC-001")).thenReturn(Optional.of(account));

// 3. Verify how it was called (after exercising the unit under test)
verify(repo).findById("ACC-001");
verify(repo, times(1)).save(any(Account.class));
verify(repo, never()).delete(any(Account.class));
```

Mockito Annotations

- Cleaner way to wire mocks
 - `@ExtendWith(MockitoExtension.class)`
 - JUnit 5 extension that hooks Mockito into the test lifecycle
 - Without it, the `@Mock` and `@InjectMocks` annotations do nothing
 - Always include it
 - `@Mock`
 - Does what `mock(AccountRepository.class)` does, but on a field
 - Mockito creates a fresh mock before each test (so tests don't share state)
 - The reset-per-test behavior is exactly what you want
 - It's why we never need to write teardown code.

```
@ExtendWith(MockitoExtension.class)
class TransferServiceTest {

    @Mock
    private AccountRepository accountRepository;

    @Mock
    private NotificationService notificationService;

    @InjectMocks
    private TransferService transferService;

    @Test
    void transferUpdatesAccount() {
        // accountRepository, notificationService are auto-created mocks
        // transferService is auto-constructed with those mocks injected
        ...
    }
}
```

Mockito Annotations

- Cleaner way to wire mocks
 - `@InjectMocks`
 - Instructs Mockito to construct the annotated object using the test's other mocks for its dependencies
 - Mockito tries constructor injection first, then setter injection, then field injection
 - For a class like `TransferService` that takes its collaborators in the constructor, Mockito picks the constructor and supplies the mocks.
 - A subtle point
 - `@InjectMocks` will not fail if some constructor parameters can't be matched to a mock, It just leaves them null
 - Occasionally a source of confusing test failures
 - You'll get a `NullPointerException` at runtime, not at test setup
 - The fix is to add the missing `@Mock` or to construct the unit under test manually in `@BeforeEach`

```
@ExtendWith(MockitoExtension.class)
class TransferServiceTest {

    @Mock
    private AccountRepository accountRepository;

    @Mock
    private NotificationService notificationService;

    @InjectMocks
    private TransferService transferService;

    @Test
    void transferUpdatesAccount() {
        // accountRepository, notificationService are auto-created mocks
        // transferService is auto-constructed with those mocks injected
        ...
    }
}
```

Stubbing Methods

- `when()` / `thenReturn()` and variants
 - The multiple consecutive returns pattern
 - For testing code that calls the same method repeatedly and expects different results each time
 - A retry loop, for example: first call fails, second call succeeds.
 - The chained `thenReturn` calls let you express this.
 - `thenThrow`
 - Essential for testing error paths
 - Use `thenThrow` to drive the unit under test into its error-handling code

```
// Single return value
when(repo.findById("ACC-001")).thenReturn(Optional.of(account));

// Different return values for consecutive calls
when(repo.findAll())
    .thenReturn(List.of(a1))      // first call
    .thenReturn(List.of(a1, a2)) // second call
    .thenReturn(List.of());      // third call onward

// Exception instead of a value
when(repo.findById("BAD-ID")).thenThrow(new DataAccessException("..."));

// Computed return value (useful for complex stubbing)
when(repo.save(any(Account.class))).thenAnswer(invocation -> {
    Account input = invocation.getArgument(0);
    return new Account(input.getAccountNumber(), input.getBalance());
});
```

Stubbing Methods

- `when()` / `thenReturn()` and variants
 - `thenAnswer`
 - For cases where the return value depends on the input
 - Most of the time `thenReturn` is enough
 - Use `thenAnswer` only when you need the mock to compute something based on what it received.

```
// Single return value
when(repo.findById("ACC-001")).thenReturn(Optional.of(account));

// Different return values for consecutive calls
when(repo.findAll())
    .thenReturn(List.of(a1))      // first call
    .thenReturn(List.of(a1, a2)) // second call
    .thenReturn(List.of());      // third call onward

// Exception instead of a value
when(repo.findById("BAD-ID")).thenThrow(new DataAccessException("..."));

// Computed return value (useful for complex stubbing)
when(repo.save(any(Account.class))).thenAnswer(invocation -> {
    Account input = invocation.getArgument(0);
    return new Account(input.getAccountNumber(), input.getBalance());
});
```

Stubbing Methods

- Stubbing void methods
 - The `when().thenReturn()` syntax
 - Does not compile for void methods because there's no return type to specify
 - Mockito provides a different syntax for void methods
 - `do...().when()`, the inverse order
 - Most common case
 - `doThrow` for testing error paths
 - For example, a NotificationService that throws when SMTP is down lets you verify your code handles the exception correctly

```
// when().thenReturn() does NOT work for void methods (compiler error)
```

```
// Use doNothing() to make a void mock do nothing (this is actually  
// the default; explicit only for clarity)  
doNothing().when(notificationService).sendEmail(any(), any());
```

```
// Use doThrow() to make a void method throw  
doThrow(new NotificationException("smtp down"))  
    .when(notificationService).sendEmail(any(), any());
```

```
// Use doAnswer() for computed side effects  
doAnswer(invocation -> {  
    String to = invocation.getArgument(0);  
    sentEmails.add(to);  
    return null;  
}).when(notificationService).sendEmail(any(), any());
```


Stubbing Methods

- Stubbing void methods
 - `doNothing` is rarely needed
 - Because doing nothing is the default behavior of any mocked void method
 - Use it only when you've previously stubbed the method to throw and want to reset to no-op behavior in a specific test
 - The general pattern to remember
 - For non-void methods, `when(...).thenSomething()`
 - For void methods, `doSomething().when(...)`
 - The difference exists because of Java's syntax constraints, not because the concepts are different

```
// when().thenReturn() does NOT work for void methods (compiler error)
```

```
// Use doNothing() to make a void mock do nothing (this is actually  
// the default; explicit only for clarity)  
doNothing().when(notificationService).sendEmail(any(), any());
```

```
// Use doThrow() to make a void method throw  
doThrow(new NotificationException("smtp down"))  
    .when(notificationService).sendEmail(any(), any());
```

```
// Use doAnswer() for computed side effects  
doAnswer(invocation -> {  
    String to = invocation.getArgument(0);  
    sentEmails.add(to);  
    return null;  
}).when(notificationService).sendEmail(any(), any());
```

Argument Matchers

- Argument matchers
 - How you express "this call, with these arguments" precisely
- Three levels of specificity
 - Exact value ("ACC-001")
 - The call matches only when the argument equals "ACC-001".
 - Type matcher (anyString(), any(Account.class))
 - The call matches any argument of that type
 - Useful when you don't care about the specific value.

```
// Exact value
when(repo.findById("ACC-001")).thenReturn(account);
verify(repo).findById("ACC-001");

// "Anything of this type"
when(repo.findById(anyString())).thenReturn(account);
verify(repo).save(any(Account.class));

// Custom predicate
verify(repo).save(argThat(a -> a.getBalance().compareTo(BigDecimal.ZERO) > 0));

// Mixing matchers and literals: must use eq() for literals
verify(repo).updateBalance(eq("ACC-001"), any(BigDecimal.class));
// Wrong: verify(repo).updateBalance("ACC-001", any(BigDecimal.class));
```

Argument Matchers

- Three levels of specificity
 - Custom predicate (`argThat(...)`)
 - The call matches when your lambda returns true. Use for complex assertions on arguments
 - You cannot mix literals and matchers in the same call
 - `eq("ACC-001")` is a matcher that matches when the argument equals "ACC-001".
 - Functionally identical to passing the literal, but it satisfies Mockito's all-or-nothing rule.

```
// WRONG  
verify(repo).updateBalance("ACC-001", any(BigDecimal.class));
```

```
// RIGHT  
verify(repo).updateBalance(eq("ACC-001"), any(BigDecimal.class));
```

Capturing Arguments

- ArgumentCaptor
 - Records the actual argument passed to the mock
 - Lets you make rich AssertJ assertions on the captured value
 - Cleaner than `argThat()` for complex assertions

```
@Test
void transferSavesUpdatedSourceAccount() {
    Account from = new Account("ACC-001", new BigDecimal("100.00"));
    when(repo.findById("ACC-001")).thenReturn(Optional.of(from));

    transferService.transfer("ACC-001", "ACC-002", new BigDecimal("30.00"));

    ArgumentCaptor<Account> captor = ArgumentCaptor.forClass(Account.class);
    verify(repo, atLeastOnce()).save(captor.capture());

    Account saved = captor.getValue();
    assertThat(saved.getAccountNumber()).isEqualTo("ACC-001");
    assertThat(saved.getBalance()).isEqualTo(new BigDecimal("70.00"));
}
```

Capturing Arguments

- ArgumentCaptor
 - When the predicate inside `argThat` returns false
 - Mockito's failure message just says "the verification failed"
 - When an AssertJ assertion fails on a captured value
 - You get "expected: 70.00 but was: 100.00".
 - The diagnostic difference is significant when tests fail in CI

```
// argThat() with complex predicate -- works, but failure messages are unhelpful
verify(repo).save(argThat(a ->
    a.getAccountNumber().equals("ACC-001") &&
    a.getBalance().equals(new BigDecimal("70.00"))));

// ArgumentCaptor with AssertJ -- clearer code, better failure messages
ArgumentCaptor<Account> captor = ArgumentCaptor.forClass(Account.class);
verify(repo).save(captor.capture());
assertThat(captor.getValue().getAccountNumber()).isEqualTo("ACC-001");
assertThat(captor.getValue().getBalance()).isEqualTo(new BigDecimal("70.00"));
```

Capturing Arguments

- ArgumentCaptor
 - `captor.getValue()`
 - Returns the most recent captured argument
 - If the method was called multiple times, use `captor.getAllValues()` to get the list
 - `@Captor` annotation also exists for declaring captors as fields
 - Similar to `@Mock`
 - Same effect as `ArgumentCaptor.forClass(...)`, just less verbose

```
// argThat() with complex predicate -- works, but failure messages are unhelpful
verify(repo).save(argThat(a ->
    a.getAccountNumber().equals("ACC-001") &&
    a.getBalance().equals(new BigDecimal("70.00"))));

// ArgumentCaptor with AssertJ -- clearer code, better failure messages
ArgumentCaptor<Account> captor = ArgumentCaptor.forClass(Account.class);
verify(repo).save(captor.capture());
assertThat(captor.getValue().getAccountNumber()).isEqualTo("ACC-001");
assertThat(captor.getValue().getBalance()).isEqualTo(new BigDecimal("70.00"));
```

Verifying Interactions

- Asserting what was called
 - Verification is the "assert" half of mocking
 - Stubbing tells the mock how to behave
 - Verification asserts how the unit under test interacted with it.
 - The most common pattern
 - `verify(mock).method(args)`
 - Assert the call happened exactly once with the matching arguments
 - The default count of one is usually right for unit tests
 - A service method should call its repository the right number of times

```
// Called exactly once (the default)
verify(repo).findById("ACC-001");

// Called a specific number of times
verify(repo, times(2)).findById(any());
verify(repo, atLeast(1)).save(any());
verify(repo, atMost(3)).findById(any());

// Never called
verify(repo, never()).delete(any());

// No interactions at all with this mock
verifyNoInteractions(notificationService);

// Specific call order
InOrder order = inOrder(repo, notificationService);
order.verify(repo).save(any());
order.verify(notificationService).sendEmail(any(), any());
```


Verifying Interactions

- Asserting what was called
 - `times(n)`, `atLeast(n)`, `atMost(n)`
 - Gives specific count matching
 - Useful when the unit under test legitimately calls a method multiple times (a retry loop, a batch operation)
 - `never()`
 - Useful for negative assertions: "this code path should NOT trigger a notification"
 - It's how you express "the unit under test made the right decision NOT to do something"

```
// Called exactly once (the default)
verify(repo).findById("ACC-001");

// Called a specific number of times
verify(repo, times(2)).findById(any());
verify(repo, atLeast(1)).save(any());
verify(repo, atMost(3)).findById(any());

// Never called
verify(repo, never()).delete(any());

// No interactions at all with this mock
verifyNoInteractions(notificationService);

// Specific call order
InOrder order = inOrder(repo, notificationService);
order.verify(repo).save(any());
order.verify(notificationService).sendEmail(any(), any());
```

Verifying Interactions

- Asserting what was called
 - `verifyNoInteractions`
 - Form for "this mock should have been ignored entirely"
 - Useful when you want to verify that an unhappy path did not, for example, send any emails
 - `inOrder`
 - For cases where the order of calls matters
 - For example, "save the account first, then send the notification"
 - If the order is reversed, a failure during notification would leave inconsistent state
 - The `inOrder` verification asserts the sequence.

```
// Called exactly once (the default)
verify(repo).findById("ACC-001");

// Called a specific number of times
verify(repo, times(2)).findById(any());
verify(repo, atLeast(1)).save(any());
verify(repo, atMost(3)).findById(any());

// Never called
verify(repo, never()).delete(any());

// No interactions at all with this mock
verifyNoInteractions(notificationService);

// Specific call order
InOrder order = inOrder(repo, notificationService);
order.verify(repo).save(any());
order.verify(notificationService).sendEmail(any(), any());
```

Example

- Line by line
 - @ExtendWith(MockitoExtension.class) activates Mockito for the test class
 - @Mock creates the accountRepository mock automatically
 - @InjectMocks constructs transferService using the mock as its dependency
 - The Arrange section creates input data and stubs the repository's behavior
 - The Act section calls the unit under test
 - The Assert section verifies the repository was called with the correct updated state

```
@ExtendWith(MockitoExtension.class)
class TransferServiceTest {

    @Mock
    private AccountRepository accountRepository;

    @InjectMocks
    private TransferService transferService;

    @Test
    void transferCallsRepositoryWithUpdatedBalances() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));
        when(accountRepository.findById("ACC-001")).thenReturn(Optional.of(from));
        when(accountRepository.findById("ACC-002")).thenReturn(Optional.of(to));

        transferService.transfer("ACC-001", "ACC-002", new BigDecimal("30.00"));

        verify(accountRepository).save(argThat(a ->
            a.getAccountNumber().equals("ACC-001") &&
            a.getBalance().equals(new BigDecimal("70.00"))));
    }
}
```

Anti-Patterns

- Anti-Patterns to avoid
 - Over-mocking
 - Mocking five collaborators to test one method means the unit under test is doing too much
 - Mocking value objects
 - Never mock String, BigDecimal, LocalDate, your own DTOs; just construct them
 - Mocking what you don't own
 - Don't mock JDBC, Spring framework classes, JPA; use them or use Spring Boot Test

```
@ExtendWith(MockitoExtension.class)
class TransferServiceTest {

    @Mock
    private AccountRepository accountRepository;

    @InjectMocks
    private TransferService transferService;

    @Test
    void transferCallsRepositoryWithUpdatedBalances() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));
        when(accountRepository.findById("ACC-001")).thenReturn(Optional.of(from));
        when(accountRepository.findById("ACC-002")).thenReturn(Optional.of(to));

        transferService.transfer("ACC-001", "ACC-002", new BigDecimal("30.00"));

        verify(accountRepository).save(argThat(a ->
            a.getAccountNumber().equals("ACC-001") &&
            a.getBalance().equals(new BigDecimal("70.00"))));
    }
}
```

Anti-Patterns

- Anti-Patterns to avoid
 - Brittle interaction tests
 - Verifying every method call is implementation-locked; verify outcomes instead
 - Thread.sleep and verify race conditions
 - If you need to wait for async behavior, use proper synchronization
 - Use the timeout and other features of Mockito

```
@ExtendWith(MockitoExtension.class)
class TransferServiceTest {

    @Mock
    private AccountRepository accountRepository;

    @InjectMocks
    private TransferService transferService;

    @Test
    void transferCallsRepositoryWithUpdatedBalances() {
        Account from = new Account("ACC-001", new BigDecimal("100.00"));
        Account to = new Account("ACC-002", new BigDecimal("0.00"));
        when(accountRepository.findById("ACC-001")).thenReturn(Optional.of(from));
        when(accountRepository.findById("ACC-002")).thenReturn(Optional.of(to));

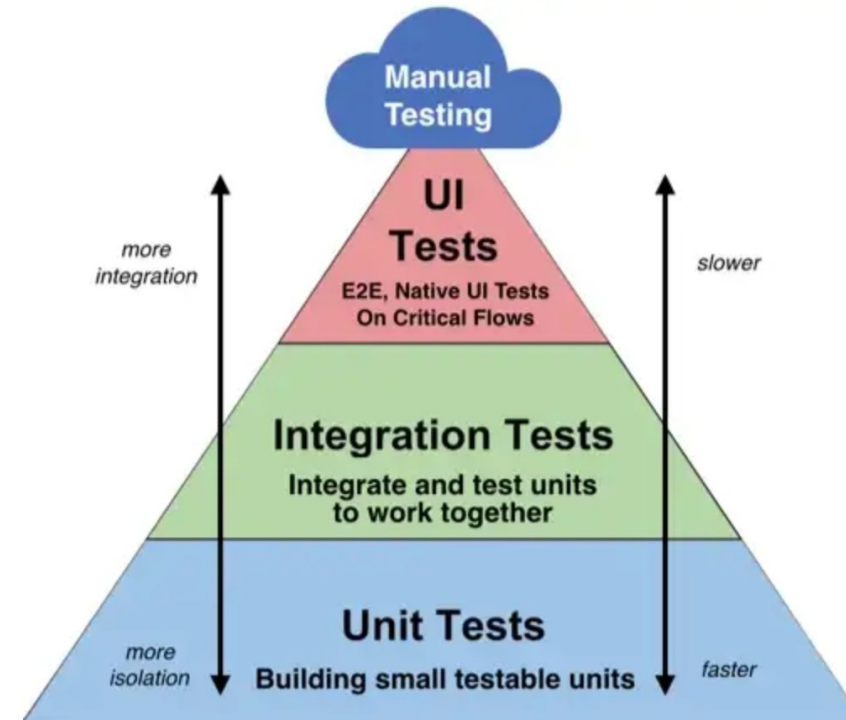
        transferService.transfer("ACC-001", "ACC-002", new BigDecimal("30.00"));

        verify(accountRepository).save(argThat(a ->
            a.getAccountNumber().equals("ACC-001") &&
            a.getBalance().equals(new BigDecimal("70.00"))));
    }
}
```

Spring Boot Integration Testing

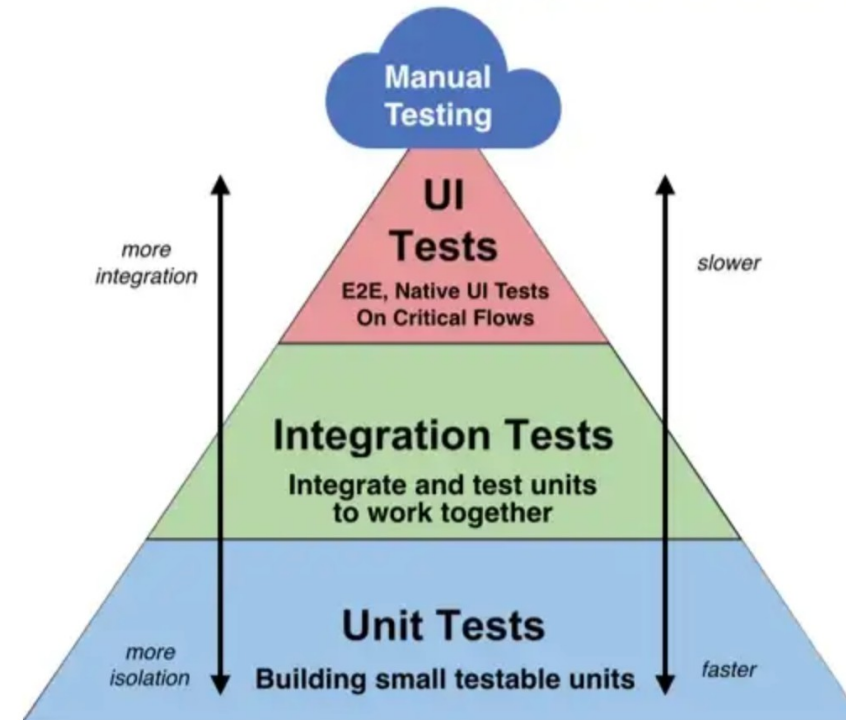
Integration Test

- Beyond single-class tests
 - Unit test
 - One class, all dependencies mocked
 - Integration test
 - Multiple components working together as Spring wires them
- In Spring
 - integration tests load some or all of the application context
 - Catches bugs at the seams
 - Serialization, security filters, request mapping, validation
 - Costs more
 - Slower startup, more setup but important when wiring matters



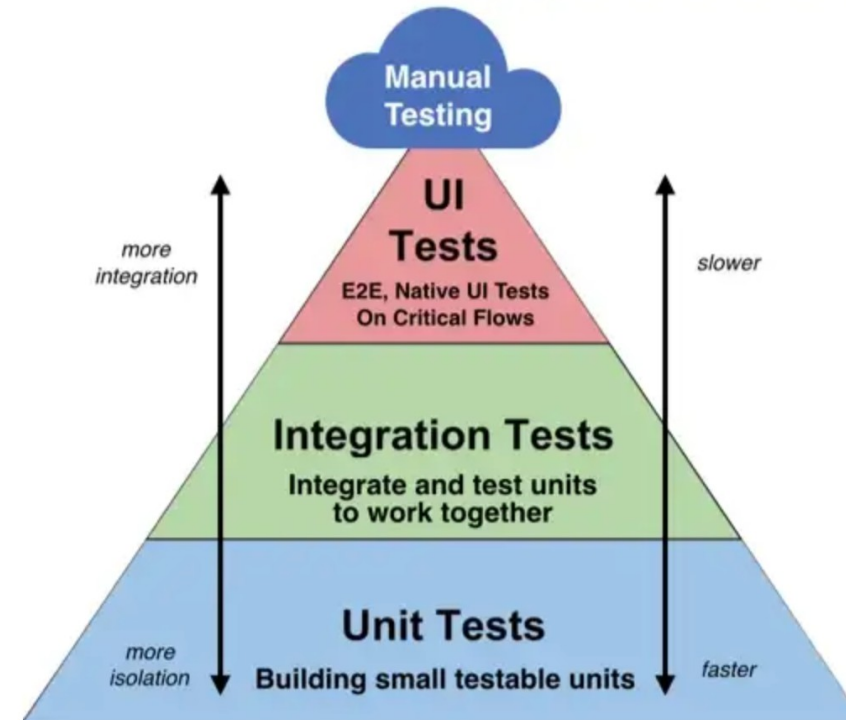
Integration Test

- The integration test catches bugs that the unit test cannot
 - A `@RequestMapping` annotation that's missing or wrong
 - A JSON serialization issue, like a field named differently than expected
 - A security configuration that allows or denies the wrong requests
 - A validation annotation that fires or fails to fire
 - An exception handler that maps the wrong status code



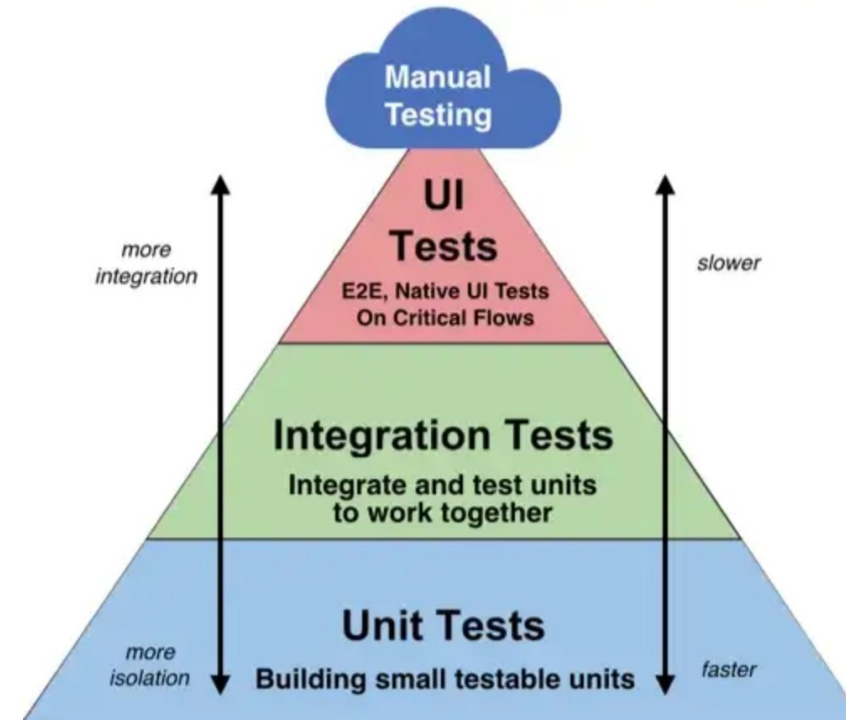
Integration Test

- Integration bugs
 - Common in Spring code
 - Because Spring does so much through annotations and configuration
 - A class can be unit-test-correct and still be badly configured in a way that breaks at the framework level
 - Integration tests close that gap.
 - The trade-off
 - Integration tests are slower and more complex to set up
 - But catches integration errors before they show up in production
 - Most of your tests should be unit tests, with a smaller layer of integration tests for the wiring



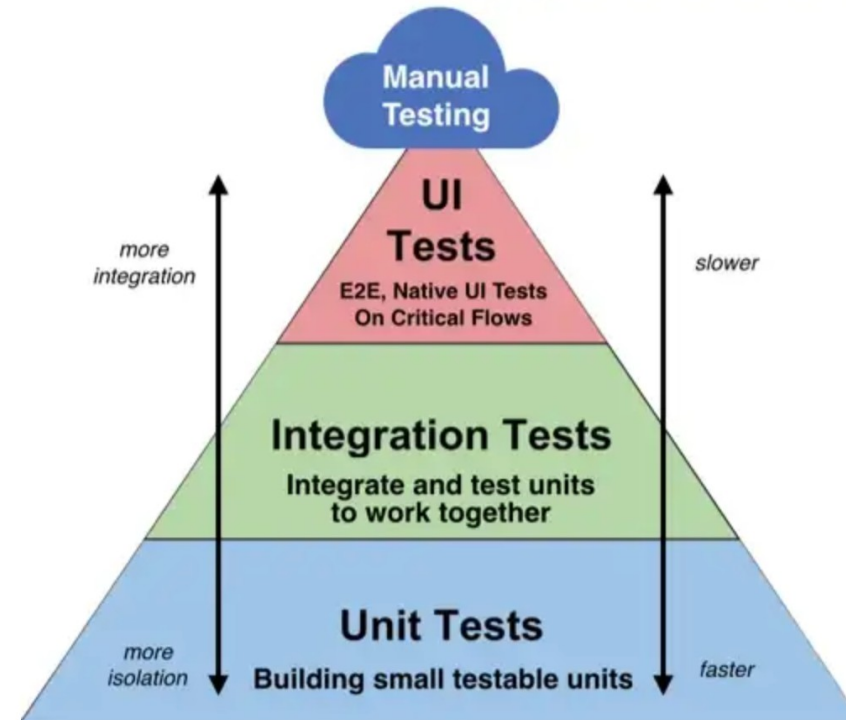
Spring Test Slices

- Loading only what is needed
 - `@SpringBootTest`
 - Loads the entire application context
 - Slowest but most realistic
 - `@WebMvcTest`
 - Loads MVC infrastructure plus one controller
 - Fast, web-focused
 - `@DataJpaTest`
 - Loads JPA infrastructure plus repositories
 - Fast, data-focused



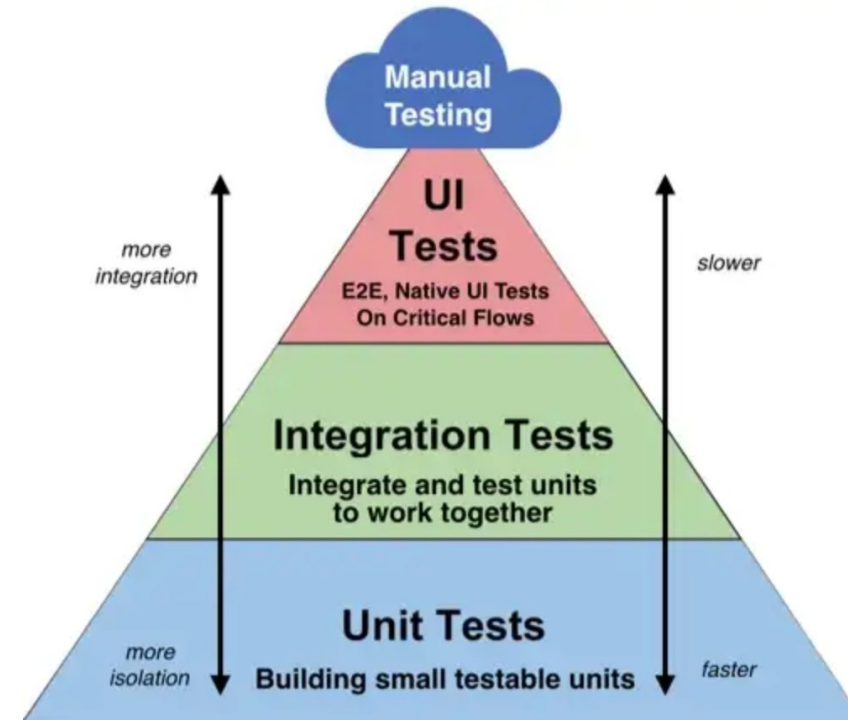
Spring Test Slices

- Loading only what is needed
 - `@JsonTest`
 - Loads JSON marshalling only
 - Very fast, narrow
 - `@RestClientTest`
 - Loads REST client infrastructure
 - Very fast, for testing outbound calls
 - The slice annotations exist to keep tests fast by NOT loading unnecessary beans



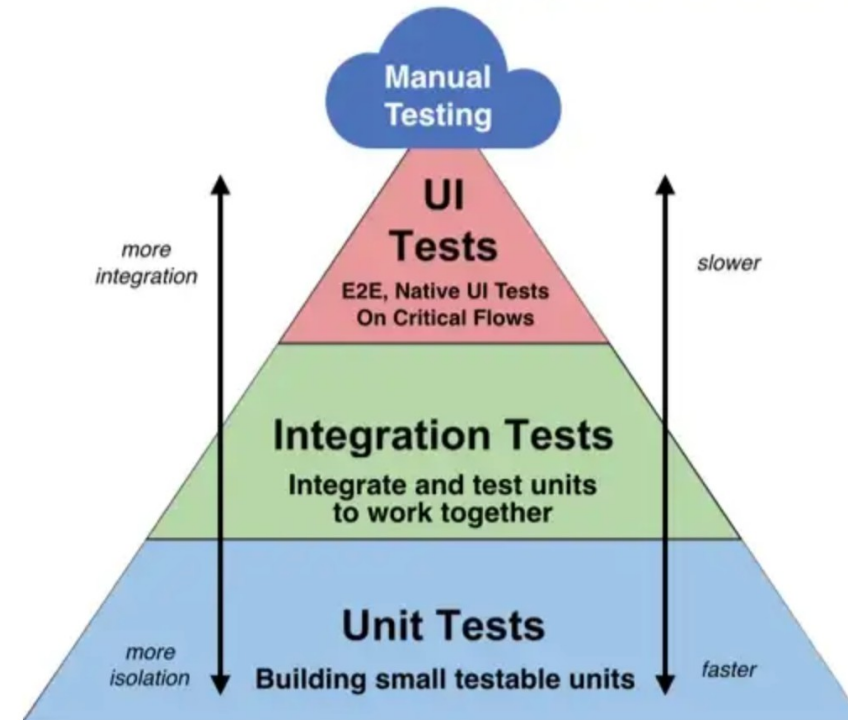
Spring Test Slices

- **@WebMvcTest**
 - Loads
 - `@Controller` and `@ControllerAdvice` beans
 - The `MockMvc` infrastructure
 - JSON converters, security configuration
 - Everything needed to test the web layer
 - It does NOT load
 - `@Service`, `@Repository`, or `@Component` beans
 - Those have to be provided as `@MockBean` in the test



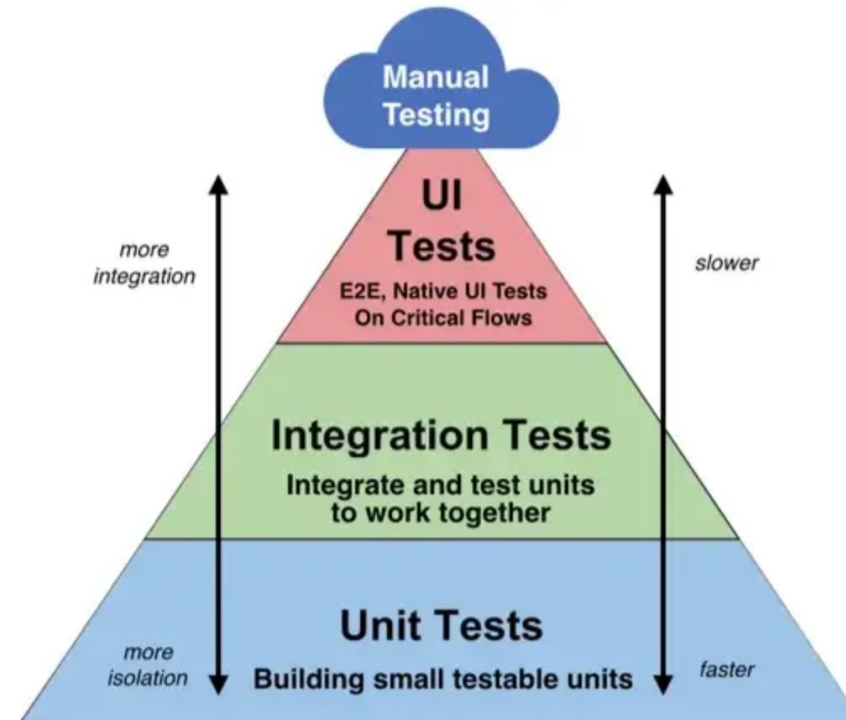
Spring Test Slices

- @DataJpaTest
 - Loads
 - JPA configuration, repositories, and an in-memory H2 database by default
 - It does NOT load controllers, services, or security
 - Useful for testing repository queries in isolation



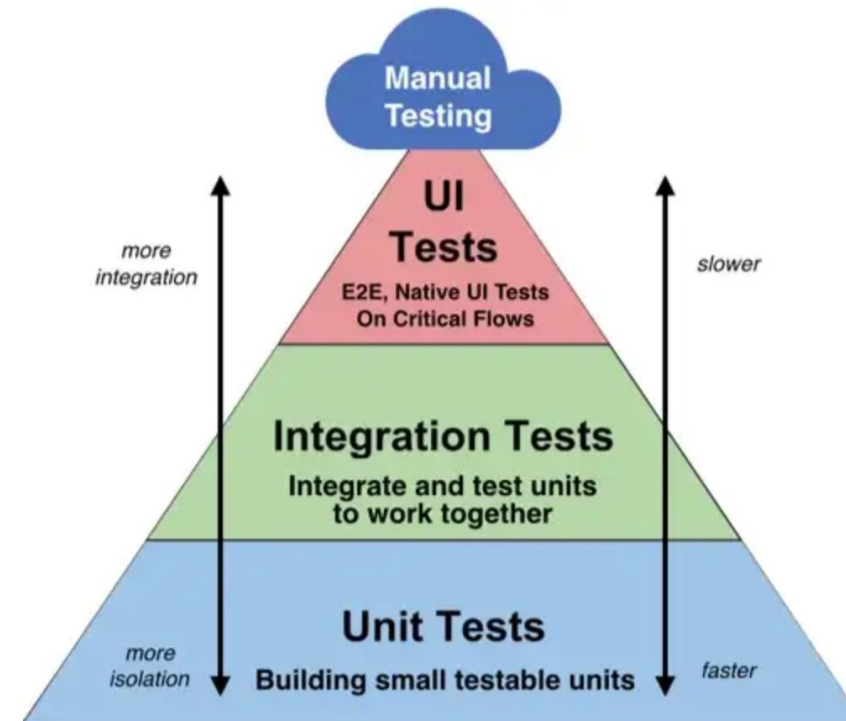
Spring Test Slices

- **@JsonTest**
 - Loads
 - Only the JSON marshalling infrastructure
 - Useful for testing custom serializers and deserializers.
- **@RestClientTest**
 - Loads
 - RestClient, WebClient, or RestTemplate configuration with a mock server
 - Useful for testing your code's outbound HTTP calls.



Spring Test Slices

- @SpringBootTest
 - Loads everything
 - Used for end to end testing.



@WebMvcTest

- Testing controllers in isolation
 - `@WebMvcTest(SomeController.class)`
 - Loads only that controller plus MVC infrastructure
 - `@MockBean`
 - Provides mocks for the controller's dependencies
 - Spring wires them in
 - `MockMvc` performs requests without starting a real server

```
@WebMvcTest(AccountController.class)
class AccountControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private AccountService accountService;

    @Test
    void getAccountsReturnsList() throws Exception {
        when(accountService.findAll()).thenReturn(List.of(
            new Account("ACC-001", new BigDecimal("100.00"))));

        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.accountNumber").value("ACC-001"));
    }
}
```

@WebMvcTest

- @WebMvcTest(AccountController.class)
 - Tells Spring:
 - “Load the MVC infrastructure plus the AccountController
 - “Don't load any other controllers, services, or repositories.”
 - The argument is optional
 - Without it, Spring loads all controllers
 - Which is sometimes useful but usually slower than necessary

```
@WebMvcTest(AccountController.class)
class AccountControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private AccountService accountService;

    @Test
    void getAccountsReturnsList() throws Exception {
        when(accountService.findAll()).thenReturn(List.of(
            new Account("ACC-001", new BigDecimal("100.00"))));

        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.accountNumber").value("ACC-001"));
    }
}
```

@WebMvcTest

- @MockBean
 - Spring Boot's version of @Mock
 - The difference
 - @Mock (from Mockito) creates a plain mock that you have to wire up manually
 - @MockBean creates a mock AND registers it in the Spring context
 - So any bean that depends on AccountService gets the mock injected automatically
 - In this test, the AccountController depends on AccountService
 - The @MockBean ensures Spring injects the mock when constructing the controller.

```
@WebMvcTest(AccountController.class)
class AccountControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private AccountService accountService;

    @Test
    void getAccountsReturnsList() throws Exception {
        when(accountService.findAll()).thenReturn(List.of(
            new Account("ACC-001", new BigDecimal("100.00"))));

        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$[0].accountNumber").value("ACC-001"));
    }
}
```

@WebMvcTest

- MockMvc
 - Spring testing tool for performing HTTP requests against controllers
 - Without actually starting an HTTP server
 - It builds the request
 - Runs it through the dispatcher servlet
 - Including security filters, request mapping, JSON marshalling
 - Returns the response for inspection
 - Fast because no real network is involved

```
@WebMvcTest(AccountController.class)
class AccountControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private AccountService accountService;

    @Test
    void getAccountsReturnsList() throws Exception {
        when(accountService.findAll()).thenReturn(List.of(
            new Account("ACC-001", new BigDecimal("100.00"))));

        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.accountNumber").value("ACC-001"));
    }
}
```

@WebMvcTest

- MockMvc
 - Has a fluent builder for constructing every HTTP method
 - The methods (get, post, put, delete, patch) are statically imported from `MockMvcRequestBuilders`
- Common builder methods
 - `.accept(...)`
 - Sets the Accept header
 - What response format the client wants
 - `.contentType(...)`
 - Sets the Content-Type of the request body

```
mockMvc.perform(get("/api/accounts/ACC-001")
    .accept(MediaType.APPLICATION_JSON)
    .andExpect(status().isOk());

mockMvc.perform(post("/api/transfers")
    .contentType(MediaType.APPLICATION_JSON)
    .content("""
        {
            "fromAccountNumber": "ACC-001",
            "toAccountNumber": "ACC-002",
            "amount": 30.00
        }
        """))
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.transactionId").exists());

mockMvc.perform(delete("/api/accounts/ACC-001"))
    .andExpect(status().isNoContent());
```

@WebMvcTest

- Common builder methods

- `.content(...)`
 - Sets the request body
 - Typically JSON as a string
- `.param("key", "value")`
 - Adds a query parameter
- `.header("Name", "value")`
 - Adds a request header
- `.cookie(...)`
 - Adds a cookie

```
mockMvc.perform(get("/api/accounts/ACC-001")
    .accept(MediaType.APPLICATION_JSON)
    .andExpect(status().isOk());

mockMvc.perform(post("/api/transfers")
    .contentType(MediaType.APPLICATION_JSON)
    .content("""
        {
            "fromAccountNumber": "ACC-001",
            "toAccountNumber": "ACC-002",
            "amount": 30.00
        }
        """))
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.transactionId").exists());

mockMvc.perform(delete("/api/accounts/ACC-001"))
    .andExpect(status().isNoContent());
```


@WebMvcTest

- Common assertions

- `status().isOk()` / `isCreated()` / `isNoContent()` / `isNotFound()` etc.
 - HTTP status code
- `header().string("Location", "/api/accounts/ACC-001")`
 - Response header
- `jsonPath(...)`
 - Assert on JSON body content
- `content().string("expected text")`
 - Raw response body
- `content().json("...")`
 - Whole-body JSON comparison (lenient by default)

```
mockMvc.perform(get("/api/accounts/ACC-001")
    .accept(MediaType.APPLICATION_JSON))
    .andExpect(status().isOk());

mockMvc.perform(post("/api/transfers")
    .contentType(MediaType.APPLICATION_JSON)
    .content("""
        {
            "fromAccountNumber": "ACC-001",
            "toAccountNumber": "ACC-002",
            "amount": 30.00
        }
        """))
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.transactionId").exists());

mockMvc.perform(delete("/api/accounts/ACC-001"))
    .andExpect(status().isNoContent());

MvcResult result = mockMvc.perform(get("/api/accounts"))
    .andExpect(status().isOk())
    .andReturn();
String body = result.getResponse().getContentAsString();
assertThat(body).contains("ACC-001");
```

@SpringBootTest

- When full context is needed
 - @SpringBootTest
 - Loads the full application context
 - Every @Service, @Repository, etc.
 - @AutoConfigureMockMvc
 - Adds MockMvc support
 - Which @WebMvcTest includes by default
 - Slower startup, but catches integration issues across all layers

```
@SpringBootTest
@AutoConfigureMockMvc
class TransferEndToEndTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private DownstreamBankClient downstreamBankClient;

    @Test
    void transferCallsDownstreamWithCorrectPayload() throws Exception {
        when(downstreamBankClient.execute(any())).thenReturn(success());

        mockMvc.perform(post("/api/transfers")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                { "fromAccountNumber": "ACC-001", ... }
            """))
            .andExpect(status().isCreated());

        verify(downstreamBankClient).execute(argThat(...));
    }
}
```

@SpringBootTest

- Right choice when
 - The integration BETWEEN layers is what you want to test
 - Eg. A POST
 - Triggers a service call
 - Which triggers a repository save
 - Which fires an event, which triggers a notification
 - That whole chain only runs end-to-end with the full context

```
@SpringBootTest
@AutoConfigureMockMvc
class TransferEndToEndTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private DownstreamBankClient downstreamBankClient;

    @Test
    void transferCallsDownstreamWithCorrectPayload() throws Exception {
        when(downstreamBankClient.execute(any())).thenReturn(success());

        mockMvc.perform(post("/api/transfers")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                { "fromAccountNumber": "ACC-001", ... }
            """))
            .andExpect(status().isCreated());

        verify(downstreamBankClient).execute(argThat(...));
    }
}
```

@SpringBootTest

- Right choice when
 - Eg. A scheduled job that depends on multiple beans being properly wired together
 - Eg. A controller
 - That uses Spring's content negotiation to serve different formats based on the Accept header
 - The full content-negotiation infrastructure only loads with `@SpringBootTest`

```
@SpringBootTest
@AutoConfigureMockMvc
class TransferEndToEndTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private DownstreamBankClient downstreamBankClient;

    @Test
    void transferCallsDownstreamWithCorrectPayload() throws Exception {
        when(downstreamBankClient.execute(any())).thenReturn(success());

        mockMvc.perform(post("/api/transfers")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                { "fromAccountNumber": "ACC-001", ... }
            """))
            .andExpect(status().isCreated());

        verify(downstreamBankClient).execute(argThat(...));
    }
}
```

@SpringBootTest

- @AutoConfigureMockMvc
 - Adds MockMvc support to a @SpringBootTest
 - @WebMvcTest already includes it
 - @SpringBootTest does not by default
 - If you want to use MockMvc with the full context, add this annotation
 - The @MockBean pattern still works inside @SpringBootTest
 - You can mock specific beans even with the full context loaded
 - Useful for replacing the downstream client which would call a real external service
 - While leaving everything else real

```
@SpringBootTest
@AutoConfigureMockMvc
class TransferEndToEndTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private DownstreamBankClient downstreamBankClient;

    @Test
    void transferCallsDownstreamWithCorrectPayload() throws Exception {
        when(downstreamBankClient.execute(any())).thenReturn(success());

        mockMvc.perform(post("/api/transfers")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                { "fromAccountNumber": "ACC-001", ... }
            """))
            .andExpect(status().isCreated());

        verify(downstreamBankClient).execute(argThat(...));
    }
}
```

Clients

- **TestRestTemplate**
 - Synchronous, blocking, simple API
 - For traditional servlet apps
- **WebTestClient**
 - Reactive, non-blocking
 - Required for WebFlux apps, optional otherwise
- Both require
 - `webEnvironment = RANDOM_PORT`
 - And a real server

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class AccountIntegrationTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Test
    void getAccountsRequiresAuthentication() {
        ResponseEntity<String> response = restTemplate.getForEntity(
            "/api/accounts", String.class);

        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.UNAUTHORIZED)
    }
}
```

Clients

- Both produce the same test
 - When to use these instead of MockMvc
 - Any time you need real network behavior
 - Testing connection timeout handling
 - Testing with multiple concurrent requests
 - Testing retry logic that depends on actual TCP behavior.
 - For most controller tests, MockMvc is faster and sufficient

```
// TestRestTemplate -- direct, synchronous
ResponseEntity<Account> response = restTemplate.getForEntity(
    "/api/accounts/ACC-001", Account.class);
assertThat(response.getStatusCode()).isEqualTo(HttpStatus.OK);
assertThat(response.getBody().getAccountNumber()).isEqualTo("ACC-001");

// WebClient -- fluent, chainable
webTestClient.get().uri("/api/accounts/ACC-001")
    .exchange()
    .expectStatus().isOk()
    .expectBody(Account.class)
    .value(account -> assertThat(account.getAccountNumber()).isEqualTo("ACC-001"));
```


OAuth-Protected Endpoints

- Authenticating without a real auth server
 - `oauth2Login()`
 - Simulates a logged-in OAuth user
 - For OAuth client apps
 - `jwt()`
 - Simulates a request with a valid JWT
 - For resource servers

```
import static org.springframework.security.test.web.servlet.request
    .SecurityMockMvcRequestPostProcessors.*;

@WebMvcTest(AccountController.class)
class AccountControllerSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void unauthenticatedRequestReturns401() throws Exception {
        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isUnauthorized());
    }

    @Test
    void authenticatedRequestSucceeds() throws Exception {
        mockMvc.perform(get("/api/accounts")
            .with(oauth2Login()))
            .andExpect(status().isOk());
    }
}
```

OAuth-Protected Endpoints

- Authenticating without a real auth server
 - `httpBasic()`
 - Simulates basic auth credentials
 - `user("alice").roles("HR_MANAGER")`
 - Simulates a logged-in user with roles

```
import static org.springframework.security.test.web.servlet.request
    .SecurityMockMvcRequestPostProcessors.*;

@WebMvcTest(AccountController.class)
class AccountControllerSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void unauthenticatedRequestReturns401() throws Exception {
        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isUnauthorized());
    }

    @Test
    void authenticatedRequestSucceeds() throws Exception {
        mockMvc.perform(get("/api/accounts")
            .with(oauth2Login()))
            .andExpect(status().isOk());
    }
}
```

OAuth-Protected Endpoints

- The wrong approach
 - Spin up a real auth server in the test
 - Perform a real OAuth flow
 - Get a real token
 - Send it with the request
 - This is slow, complex, and tests the auth server more than the application

```
import static org.springframework.security.test.web.servlet.request
    .SecurityMockMvcRequestPostProcessors.*;

@WebMvcTest(AccountController.class)
class AccountControllerSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void unauthenticatedRequestReturns401() throws Exception {
        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isUnauthorized());
    }

    @Test
    void authenticatedRequestSucceeds() throws Exception {
        mockMvc.perform(get("/api/accounts")
            .with(oauth2Login()))
            .andExpect(status().isOk());
    }
}
```

OAuth-Protected Endpoints

- The right approach
 - Use Spring Security's test post-processors to simulate an authenticated request
 - The post-processors
 - Construct the security context that would exist after a real authentication
 - without doing any real authentication
 - `oauth2Login()`
 - Right post-processor for OAuth client applications like the bankbff
 - It simulates the post-login state
 - There's an `OAuth2AuthenticationToken` in the security context, the user appears to be logged in
 - The controller sees this and behaves as if authentication had happened

```
import static org.springframework.security.test.web.servlet.request
    .SecurityMockMvcRequestPostProcessors.*;

@WebMvcTest(AccountController.class)
class AccountControllerSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void unauthenticatedRequestReturns401() throws Exception {
        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isUnauthorized());
    }

    @Test
    void authenticatedRequestSucceeds() throws Exception {
        mockMvc.perform(get("/api/accounts")
            .with(oauth2Login()))
            .andExpect(status().isOk());
    }
}
```

OAuth-Protected Endpoints

- The right approach
 - `jwt()`
 - Tight post-processor for OAuth resource servers like the bankserver
 - It simulates a request with a valid JWT
 - Populating the security context with a `JwtAuthenticationToken`
 - Again, no real JWT is created, decoded, or verified. just simulated.
 - `user("alice").roles("HR_MANAGER")`
 - Simpler version for non-OAuth apps
 - It simulates form-based authentication with the given username and roles.

```
import static org.springframework.security.test.web.servlet.request
    .SecurityMockMvcRequestPostProcessors.*;

@WebMvcTest(AccountController.class)
class AccountControllerSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void unauthenticatedRequestReturns401() throws Exception {
        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isUnauthorized());
    }

    @Test
    void authenticatedRequestSucceeds() throws Exception {
        mockMvc.perform(get("/api/accounts")
            .with(oauth2Login()))
            .andExpect(status().isOk());
    }
}
```

OAuth-Protected Endpoints

- The second test:
 - .with(oauth2Login()) chained onto the request
 - Adds the simulated authentication
 - Without it, the same request fails with 401
 - With it, the request succeeds with 200.
 - Forgetting to test the unauthenticated case
 - The first test is just as important as the authenticated case
 - It verifies that security is actually enforced
 - A change that accidentally permits all requests would pass the authenticated test because it's authenticated
 - Only the unauthenticated test catch that all all requests are allowed

```
import static org.springframework.security.test.web.servlet.request
    .SecurityMockMvcRequestPostProcessors.*;

@WebMvcTest(AccountController.class)
class AccountControllerSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void unauthenticatedRequestReturns401() throws Exception {
        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isUnauthorized());
    }

    @Test
    void authenticatedRequestSucceeds() throws Exception {
        mockMvc.perform(get("/api/accounts")
            .with(oauth2Login()))
            .andExpect(status().isOk());
    }
}
```


Test Profiles

- Test profile
 - Lets you override production settings
 - e.g., real DB to in-memory
 - Property overrides for individual test classes that need specific settings
 - application-test.yml lives alongside application.yml
 - Applies when test profile is active

```
// Activate the "test" profile for this test class
@SpringBootTest
@ActiveProfiles("test")
class AccountServiceIntegrationTest { ... }

// Inline property overrides (for one test class)
@SpringBootTest(properties = {
    "app.bank.timeout=100ms",
    "app.bank.retry-count=0"
})
class FastFailureTest { ... }

// Properties file: src/test/resources/application-test.yml
// Loaded automatically when the "test" profile is active
```


Test Profiles

- Real applications
 - Often need different configuration in tests than in production
 - Examples
 - Production uses PostgreSQL; tests use H2 in-memory
 - Production uses real OAuth credentials; tests use mocked authentication
 - Production has 30-second timeouts; tests need 100ms timeouts to keep test runs fast
 - Production sends real emails; tests use a mock email sender

```
// Activate the "test" profile for this test class
@SpringBootTest
@ActiveProfiles("test")
class AccountServiceIntegrationTest { ... }

// Inline property overrides (for one test class)
@SpringBootTest(properties = {
    "app.bank.timeout=100ms",
    "app.bank.retry-count=0"
})
class FastFailureTest { ... }

// Properties file: src/test/resources/application-test.yml
// Loaded automatically when the "test" profile is active
```

Test Profiles

- Three places configuration can come from in tests
 - `application-test.yml` in `src/test/resources`
 - Loaded automatically when the test profile is active
 - Use this for settings that apply to ALL tests
 - `@ActiveProfiles("test")` on a test class
 - Activates the test profile for that class
 - Without this, the profile is not active and `application-test.yml` is not loaded.

```
// Activate the "test" profile for this test class
@SpringBootTest
@ActiveProfiles("test")
class AccountServiceIntegrationTest { ... }

// Inline property overrides (for one test class)
@SpringBootTest(properties = {
    "app.bank.timeout=100ms",
    "app.bank.retry-count=0"
})
class FastFailureTest { ... }

// Properties file: src/test/resources/application-test.yml
// Loaded automatically when the "test" profile is active
```

Test Profiles

- Three places configuration can come from in tests
 - `@SpringBootTest(properties = {...})` on a test class
 - Inline property overrides for tests that need specific settings without modifying any file
 - Useful for one-off scenarios.

```
// Activate the "test" profile for this test class
@SpringBootTest
@ActiveProfiles("test")
class AccountServiceIntegrationTest { ... }
```

```
// Inline property overrides (for one test class)
@SpringBootTest(properties = {
    "app.bank.timeout=100ms",
    "app.bank.retry-count=0"
})
class FastFailureTest { ... }
```

```
// Properties file: src/test/resources/application-test.yml
// Loaded automatically when the "test" profile is active
```

Test Profiles

- Order matters
 - Inline properties override `application-test.yml`
 - Which overrides `application.yml`
 - The most specific source wins
 - A common test profile setup for the bankbff project is shown to the right
 - This avoids needing real OAuth credentials in tests
 - Combined with the `oauth2Login()` post-processor from the previous slide, tests can run without ever talking to a real OAuth server

```
# src/test/resources/application-test.yml
spring:
  security:
    oauth2:
      client:
        registration:
          bank-auth:
            client-id: test-client-id
            client-secret: test-client-secret
```

Database Isolation

- Keeping tests from polluting other tests
 - `@Transactional` on test methods
 - Changes are rolled back after each test
 - `@Sql("/test-data.sql")`
 - Runs SQL setup before each test
 - Avoid `@Transactional` if you need to test transactional behavior itself
 - For real-database testing, use `Testcontainers`
 - PostgreSQL in a Docker container per test class for example

```
@SpringBootTest
@Transactional
class AccountRepositoryTest {

    @Autowired
    private AccountRepository repo;

    @Test
    void savedAccountIsRetrievable() {
        Account a = new Account("ACC-001", new BigDecimal("100.00"));
        repo.save(a);

        assertThat(repo.findById("ACC-001")).isPresent();
        // After this test method returns, the rollback happens.
        // The next test starts with no ACC-001 in the database.
    }
}
```

Database Isolation

- `@Sql`
 - Runs SQL files before tests
 - Useful for setting up reference data that every test in the class needs
 - Place the SQL file in `src/test/resources/`
- `@Transactional`
 - Rollback is convenient but has a catch
 - Don't use it if the test itself is testing transactional behavior
 - If your code under test commits or rolls back transactions, the test's own outer transaction interferes with that
 - For these cases, manage cleanup manually or use a different isolation approach

```
@Test
@Sql("/setup-test-accounts.sql")
void findByStatusReturnsActiveAccounts() {
    // setup-test-accounts.sql ran before this test
    // It inserted accounts with various statuses
    List<Account> active = repo.findByStatus("ACTIVE");
    assertThat(active).hasSize(3);
}
```

Integration Test

- Walk through
 - `@WebMvcTest(AccountController.class)`
 - loads the AccountController and the MVC infrastructure (including security)
 - No services, no repositories.
 - `@MockBean`
 - `BankingApiClient` provides a mock for the controller's downstream collaborator
 - The controller uses this client to fetch data
 - The mock lets the test control what data comes back.

```
@WebMvcTest(AccountController.class)
class AccountControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private BankingApiClient bankingApiClient;

    @Test
    void getAccountsReturnsListWhenAuthenticated() throws Exception {
        when(bankingApiClient.getAccounts(any())).thenReturn(List.of(
            new Account("ACC-001", "CHECKING", new BigDecimal("100.00"), "ACTIVE")));

        mockMvc.perform(get("/api/accounts")
            .with(oauth2Login()))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$[0].accountNumber").value("ACC-001"));
    }

    @Test
    void getAccountsReturns401WhenUnauthenticated() throws Exception {
        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isUnauthorized());
    }
}
```


Integration Test

- The second test
 - Performs the same GET without authentication
 - Asserts the response is 401 Unauthorized
- The two tests together
 - Cover the contract from two angles
 - The endpoint works when authenticated
 - The endpoint refuses when not authenticated

```
@WebMvcTest(AccountController.class)
class AccountControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private BankingApiClient bankingApiClient;

    @Test
    void getAccountsReturnsListWhenAuthenticated() throws Exception {
        when(bankingApiClient.getAccounts(any())).thenReturn(List.of(
            new Account("ACC-001", "CHECKING", new BigDecimal("100.00"), "ACTIVE")));

        mockMvc.perform(get("/api/accounts")
            .with(oauth2Login()))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.accountNumber").value("ACC-001"));
    }

    @Test
    void getAccountsReturns401WhenUnauthenticated() throws Exception {
        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isUnauthorized());
    }
}
```

Integration Test

- A subtle point
 - `@WebMvcTest` includes Spring Security in the loaded context
 - If your security config specifies that `/api/accounts` requires authentication
 - The unauthenticated test correctly returns 401
 - If the security config (accidentally or intentionally) permits the request, the unauthenticated test returns 200
 - Which would catch a real incorrect configuration.

```
@WebMvcTest(AccountController.class)
class AccountControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private BankingApiClient bankingApiClient;

    @Test
    void getAccountsReturnsListWhenAuthenticated() throws Exception {
        when(bankingApiClient.getAccounts(any())).thenReturn(List.of(
            new Account("ACC-001", "CHECKING", new BigDecimal("100.00"), "ACTIVE")));

        mockMvc.perform(get("/api/accounts")
            .with(oauth2Login()))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.accountNumber").value("ACC-001"));
    }

    @Test
    void getAccountsReturns401WhenUnauthenticated() throws Exception {
        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isUnauthorized());
    }
}
```

Integration Test

- A subtle point
 - This is exactly the kind of bug that's hard to catch with unit tests alone
 - The controller's logic might be correct in isolation; the security config might be correct in isolation
 - Whether the security config actually applies to the controller's URL pattern
 - Only shows up in an integration test.

```
@WebMvcTest(AccountController.class)
class AccountControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private BankingApiClient bankingApiClient;

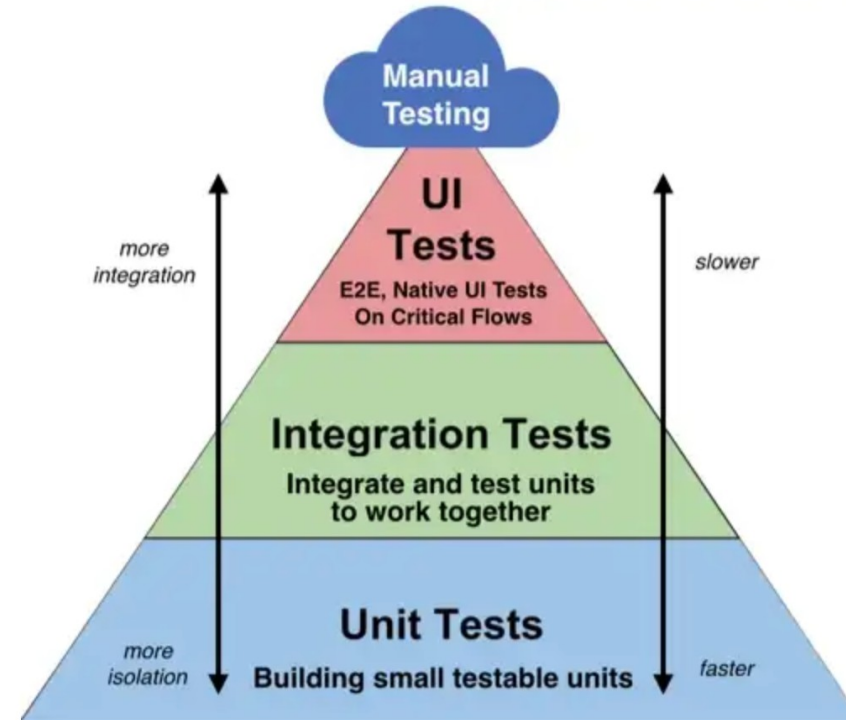
    @Test
    void getAccountsReturnsListWhenAuthenticated() throws Exception {
        when(bankingApiClient.getAccounts(any())).thenReturn(List.of(
            new Account("ACC-001", "CHECKING", new BigDecimal("100.00"), "ACTIVE")));

        mockMvc.perform(get("/api/accounts")
            .with(oauth2Login()))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$[0].accountNumber").value("ACC-001"));
    }

    @Test
    void getAccountsReturns401WhenUnauthenticated() throws Exception {
        mockMvc.perform(get("/api/accounts"))
            .andExpect(status().isUnauthorized());
    }
}
```

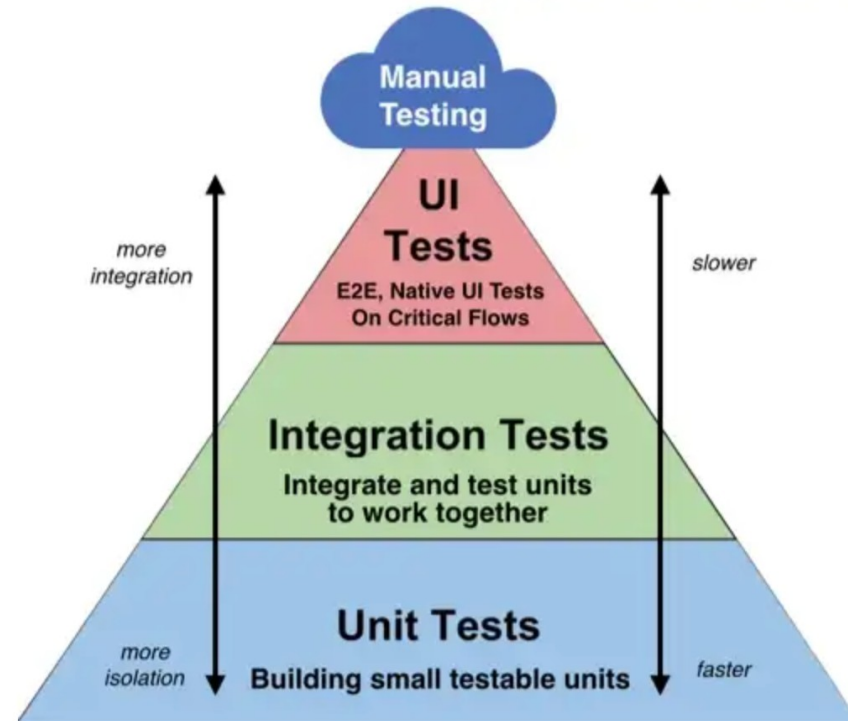
Test Pyramid Redux

- Testing effort
 - Unit tests
 - Many tests
 - Validates logic in isolation
 - Execute fast, pinpoint failures
 - Integration tests
 - Some tests
 - Test wiring, security, serialization, multi-component flows
 - End-to-end tests
 - A few tests
 - Test the system through the UI
 - Tend to be slow, fragile



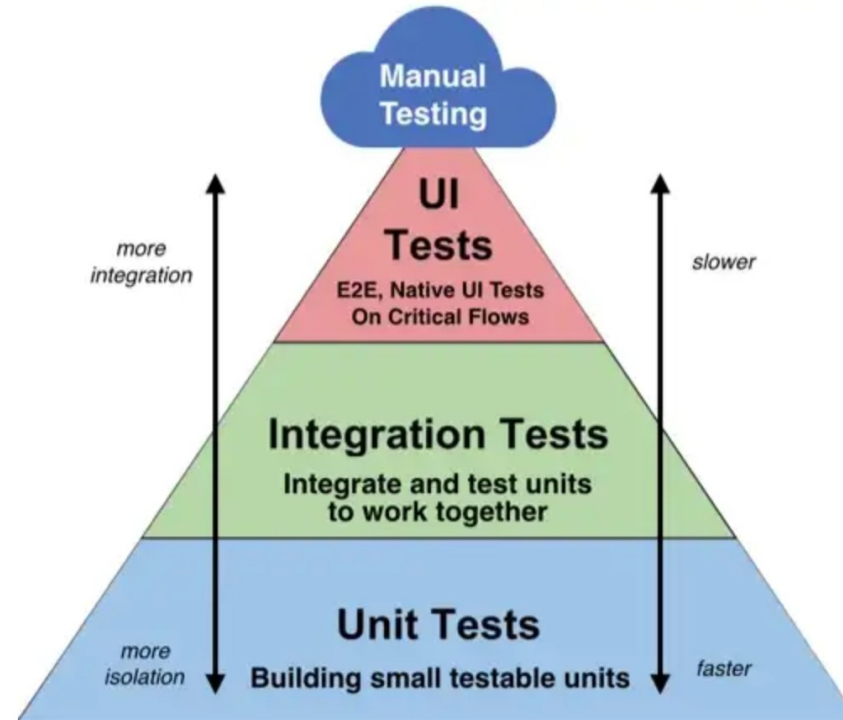
Test Pyramid Redux

- A typical Spring Boot project
 - 70% unit
 - 25% integration
 - 5% end-to-end
 - Adjust based on what kinds of bugs your project actually produces



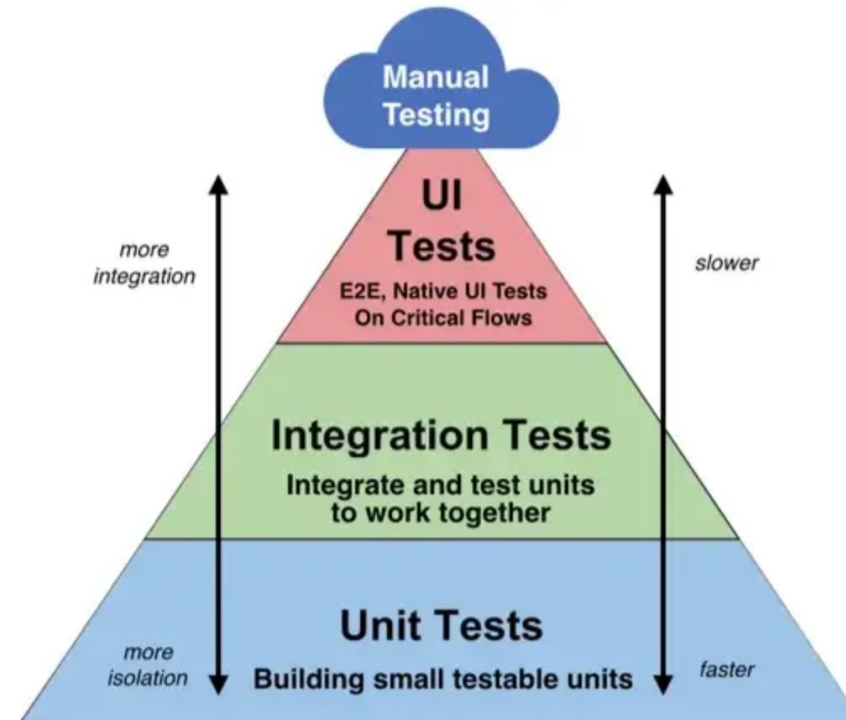
Test Pyramid Redux

- When to choose integration over unit
 - Testing security configuration
 - Authorization rules, public vs. protected endpoints
 - Testing JSON serialization
 - Custom serializers, field naming
 - Testing exception handling
 - Which exceptions map to which HTTP status codes
 - Testing request validation
 - @Valid annotations, custom validators
 - Testing the wiring between Spring beans



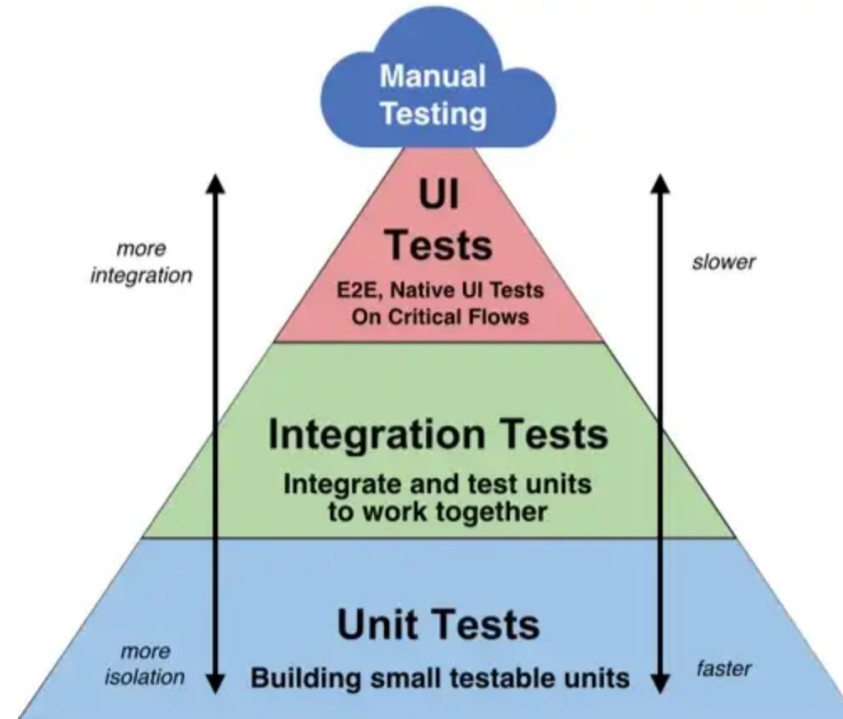
Test Pyramid Redux

- When unit tests are sufficient:
 - Pure logic
 - Calculations, transformations, validations)
 - Service classes with mockable dependencies
 - Helper utilities



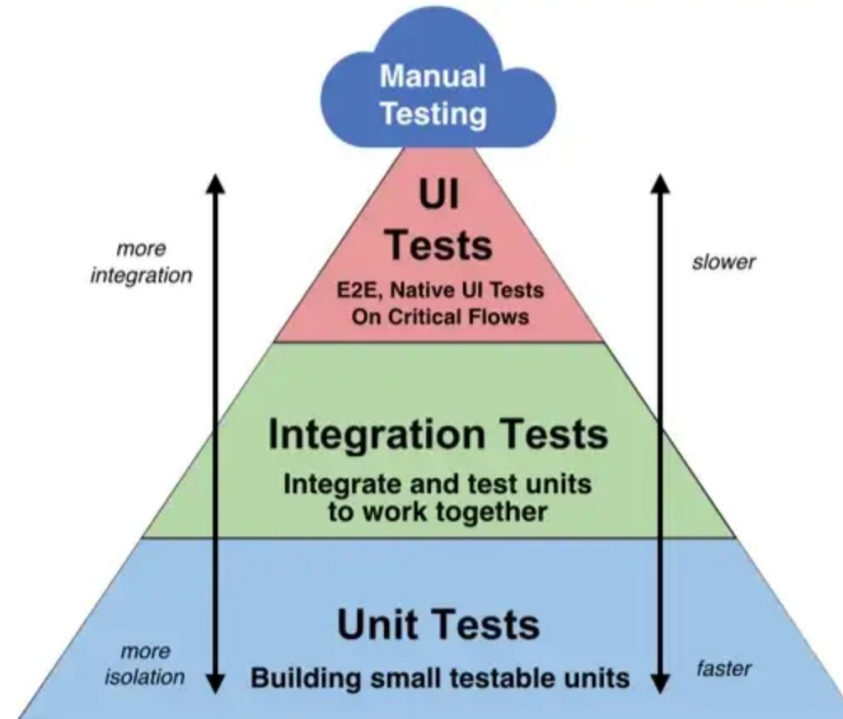
Integration Testing

- Mistakes to avoid
 - Loading too much
 - Using `@SpringBootTest` when `@WebMvcTest` would suffice
 - Tests slow for no reason
 - Not testing security
 - Skipping the unauthenticated case means you don't actually know security is enforced
 - Sharing state between tests
 - Forgetting `@Transactional` or proper teardown produces flaky tests



Integration Testing

- Mistakes to avoid
 - Using real downstream services
 - Never call real external APIs in tests; always mock at the boundary
 - Snapshot/golden tests with brittle assertions
 - Whole-body JSON comparisons break on every cosmetic change



Questions?

